



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base

Corso di Laurea Magistrale in Ingegneria Informatica

Software Architecture Design

Sistema Suggerimenti

Documentazione Gruppo D2 Task R3

Anno Accademico 2025/2026

Componenti del gruppo:

Nome

Matricola

Emanuele De Rosa

DE9000164

Antonio Giaquinto

DE9000142

Giovanni Gismondi

N46007033

Domenico Di Gennaro

DE9000051

Sommario

1. Identificazione del Team e Obiettivi del Progetto.....	3
2. Analisi dei Requisiti.....	5
2.1. Requisiti funzionali	5
2.2. Requisiti sui dati.....	6
2.3. Vincoli.....	7
2.4. Modellazione dei casi d'uso	8
2.4.1. Attori e casi d'uso	8
2.4.2. Diagramma dei casi d'uso	10
2.4.3. Scenari Casi d'Uso	10
2.5. Analisi dell'Impatto	15
3. Progettazione della soluzione	16
3.1. Decisioni di Progetto	16
3.2. Panoramica dei file modificati	18
3.3. Viste Architetture Aggornate.....	21
3.3.1. Class Diagram	21
3.3.2. Sequence Diagrams	23
3.3.3. Component Diagram	26
3.4. Modifiche ai Database	28
3.5. Issue e Anti-pattern.....	29
4. Implementazione e Struttura del Progetto.....	32
4.1. Moduli Aggiunti o Modificati	32
4.2. Refactoring e Modifiche.....	32
5. Test.....	34
5.1. Testing di Unità del microservizio T1	34
5.1.1. Architettura dei test	34
5.1.2. Unit Test Suggestion Service di T1.....	35
5.1.3. Unit Test Suggestion Controller in T1.....	39
5.2. Test di Sistema	44
6. Sviluppi Futuri e Raccomandazioni.....	57

1. Identificazione del Team e Obiettivi del Progetto

ID del Team: D2

Componenti del Team:

Nome

Mail

Emanuele De Rosa

emanuele.derosa3@studenti.unina.it

Antonio Giaquinto

antonio.giaquinto4@studenti.unina.it

Giovanni Gismondi

gio.gismondi@studenti.unina.it

Domenico Di Gennaro

dom.digennaro@studenti.unina.it

URL Versione di Partenza del Progetto:

<https://github.com/Testing-Game-SAD-2023/A13.git>

URL Repository del Progetto Consegnato:

<https://github.com/MrD0me/A13/tree/main>

Task Assegnato e approcci:

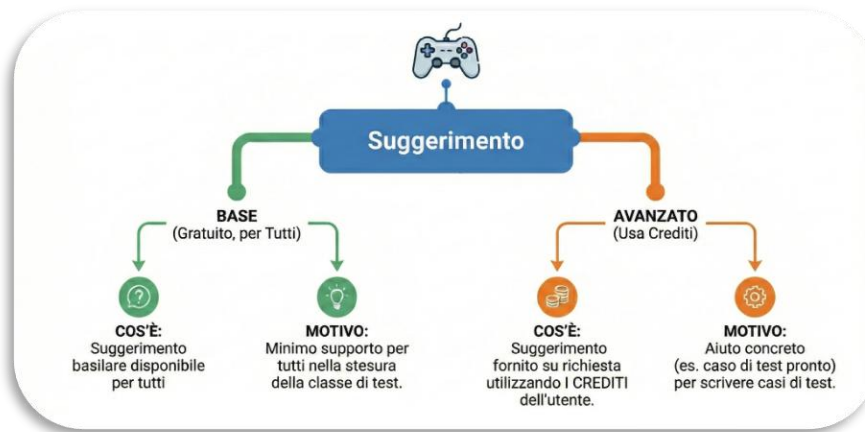
Il task assegnato consiste nell'implementare all'interno del gioco la possibilità per un giocatore di richiedere suggerimenti visualizzabili durante la scrittura del codice in una partita.

Lo sviluppo di questo task è stato condotto seguendo metodologie agili e uno sviluppo incrementale. Durante le review periodiche a cadenza bisettimanale, abbiamo presentato una versione aggiornata e funzionante del sistema. Durante suddette review abbiamo ricevuto feedback che ci hanno permesso di migliorare le funzioni già implementate e di avere spunti per la realizzazione di nuove funzionalità.

Cosa Abbiamo realizzato:

Si è ritenuto necessario implementare un pulsante che durante la partita permette di aprire un menù modale che consente di:

- Richiedere suggerimenti, sia di base (guide line) che “avanzati” (relativi ad una determinata classe).



- Visualizzare il contatore dei suggerimenti rimanenti sul numero di suggerimenti totali disponibili.
- Visualizzare uno storico dei suggerimenti richiesti

Un'ulteriore funzionalità aggiunta è la possibilità di inserire, tramite apposito pulsante, il testo del suggerimento direttamente all'interno dell'editor sottoforma di commento. Ciò è pensato per eventuali suggerimenti che consegnano delle linee di codice all'utente, come ad esempio un intero test case.

Oltre alle funzionalità principali, è stato implementato un sistema di crediti legato al profilo utente. I crediti vengono assegnati quando il giocatore vince una partita utilizzando una classe mai impiegata prima a un determinato livello di difficoltà. Questi crediti possono essere spesi per sbloccare suggerimenti avanzati, ideati per offrire un supporto superiore rispetto a quelli base.

2. Analisi dei Requisiti

In questo capitolo viene definita la struttura funzionale e tecnica necessaria alla gestione dei suggerimenti. L'analisi si articola attraverso la specifica dei requisiti (funzionali, dei dati e non funzionali), la modellazione dei casi d'uso con i relativi scenari e la definizione del modello dati. Viene poi presentata un'analisi d'impatto per valutare come l'introduzione di queste funzionalità influenzi i componenti del sistema preesistente.

2.1. Requisiti funzionali

ID	Descrizione
RF-01	Il sistema deve permettere al giocatore di visualizzare un suggerimento di tipo "Base". L'azione deve decrementare il contatore dei suggerimenti disponibili.
RF-02	Il sistema deve permettere l'acquisto di un suggerimento "Avanzato". L'operazione deve avvenire solo se il saldo crediti è sufficiente ($Saldo \geq Costo$) e deve scalare immediatamente i crediti.
RF-03	Il sistema deve mostrare in tempo reale: 1. Numero suggerimenti Base residui (calcolati secondo RD-03). 2. Numero suggerimenti Avanzati totali. 3. Saldo Crediti attuale.
RF-04	Il sistema deve mantenere una lista visibile di tutti i suggerimenti (Base e Avanzati) già sbloccati nella partita corrente, distinguendoli visivamente per tipo.
RF-05	Ad ogni richiesta (RF-01 o RF-02), il sistema deve estrarre un suggerimento casuale dal pool disponibile, escludendo tassativamente quelli già presenti nello storico (evitare duplicati)
RF-06	Il sistema deve fornire un metodo rapido (es. bottone «Inserisci») per inserire il contenuto del suggerimento (snippet di codice) nell'editor dell'utente.
RF-07	Il sistema deve accreditare una quantità definita di crediti al profilo utente al superamento della classe di test (Vittoria).

2.2. Requisiti sui dati

ID	Descrizione
RD-01	Ogni suggerimento possiede: ID (univoco), Testo /Codice, Tipo (Base/Avanzato), Difficoltà (Easy, Medium, Hard), Classe Target (0 per Base).
RD-02	Il numero di suggerimenti Base visibili è dato dalla formula: $N_Show = MIN(Limite_Difficoltà, Totale_DB)$ Dove: - <i>Limite_Difficoltà</i> = è il limite massimo calcolato in base alla difficoltà della partita - <i>Totale_DB</i> = è il conteggio totale dei suggerimenti presenti nel database afferenti a quella determinata classe e difficoltà (Si mostra il minimo tra il limite imposto dalla difficoltà e quelli realmente esistenti nel DB).
RD-03	I limiti massimi per i suggerimenti Base sono fissi: - FACILE: 10 - MEDIO: 5 - DIFFICILE: 2
RD-04	Un suggerimento, una volta erogato e aggiunto allo storico, non può essere estratto nuovamente nella stessa sessione di gioco.

2.3. Vincoli

ID	Descrizione
VN-01	Il sistema deve salvare lo storico dei suggerimenti sbloccati nel Local Storage (o Session Storage) per garantire che i dati non vadano persi in caso di refresh della pagina (F5) o ripresa della partita.
VN-02	La verifica del saldo crediti e l'estrazione casuale del suggerimento devono avvenire lato Server (Backend) per impedire manipolazioni da parte del client.
VN-03	Se la chiamata API fallisce l'interfaccia di gioco deve rimanere attiva
VN-04	In caso di errore (es. crediti insufficienti o suggerimenti esauriti), il sistema deve fornire un feedback visivo

Durante lo sviluppo del task assegnato abbiamo riscontrato diverse problematiche nell'integrare il nostro lavoro con quello del gruppo che si è occupato della migrazione del DataBase in T1 da MongoDB a MySQL e dell'inserimento dei suggerimenti dal punto di vista dell'amministratore, per questo motivo abbiamo deciso di implementare anche queste funzionalità, in modo da poter presentare una versione stabile e funzionante dell'applicazione, che permetta sia di inserire, modificare ed eliminare suggerimenti, sia di poterli visualizzare mentre si gioca.

2.4 Requisiti funzionali lato amministratore

ID	Descrizione
RFA-01	Il sistema deve permettere all'amministratore di inserire nuovi suggerimenti.
RFA-02	Il sistema deve permettere all'amministratore di visualizzare la lista dei suggerimenti presenti nel DataBase.
RFA-03	Il sistema deve permettere all'amministratore di eliminare un suggerimento dal DataBase
RFA-04	Il sistema deve permettere all'amministratore di impostare il numero di suggerimenti base disponibili in base alla difficoltà della classe

2.5 Vincoli lato amministratore

ID	Descrizione
VA-01	Il sistema imposta automaticamente la lingua del suggerimento in

	italiano quando si inserisce un nuovo suggerimento senza specificarne la lingua
VA-02	I campi classe, difficoltà, tier e testo non possono essere vuoti, nel caso l'amministratore non riempia questi campi il sistema deve mostrare un messaggio di errore

2.6 Modellazione dei casi d'uso

2.6.1 Attori e casi d'uso

Gli attori Primari individuati sono tre: Utente Registrato, Amministratore e Sistema. I casi d'uso sono stati suddivisi, attraverso le tabelle qui di seguito, in: generali, di inclusione e di estensione.

Casi d'uso:

ID	Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.
UC01	Richiesta Suggerimento BASE	Giocatore	-	Include: Visualizza suggerimento Esteso da: Inserisci Suggerimento
UC02	Richiesta suggerimento AVANZATO	Giocatore	-	Include: Visualizza suggerimento Esteso da: Inserisci Suggerimento
UC03	Visualizza Storico Suggerimenti	Giocatore	-	-
UC04	Vittoria Partita	Giocatore	-	-
UC05	Carica Suggerimento	Amministratore	-	-
UC06	Visualizza	Giocatore	-	-

	crediti disponibili			
UC07	Elimina suggerimento	Amministratore		
UC08	Visualizza lista suggerimenti	Amministratore		

Casi d'uso di inclusione:

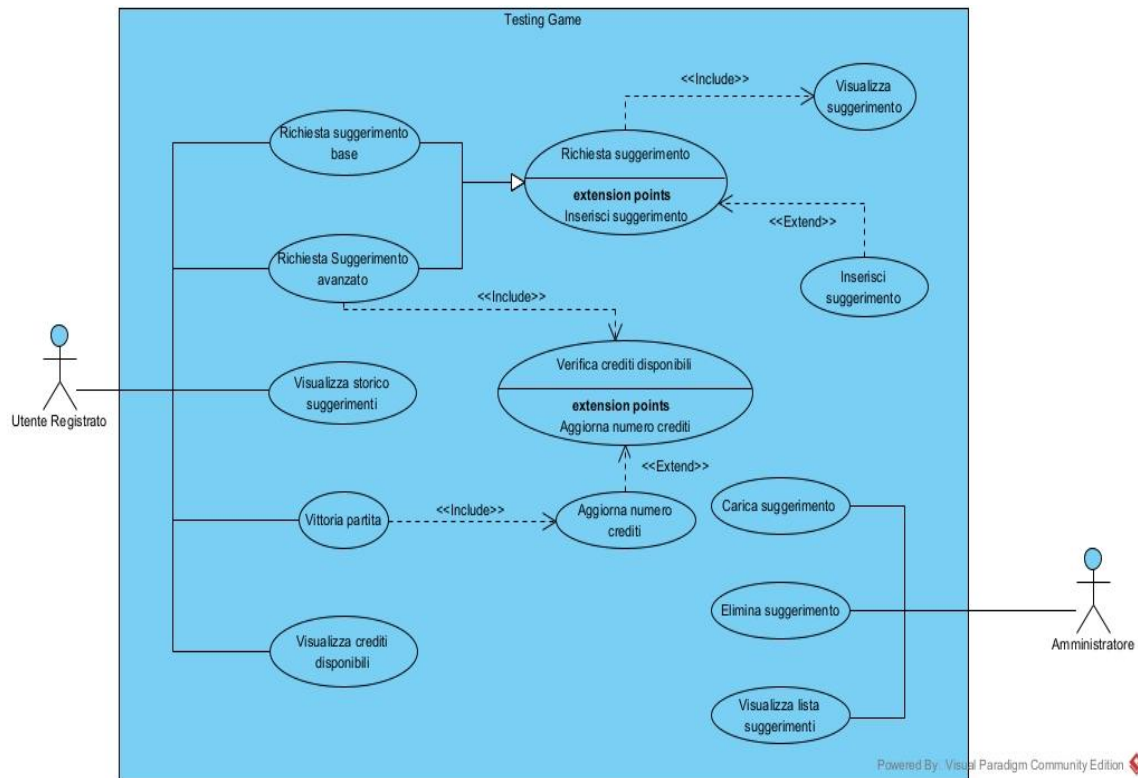
ID	Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.
UC09	Visualizza Suggerimento	-	-	-
UC10	Verifica crediti disponibili	-	-	-

Casi d'uso di estensione:

ID	Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.
UC11	Inserisci suggerimento	-	-	-
UC12	Aggiorna numero crediti	-	-	-

2.6.2 Diagramma dei casi d'uso

Partendo dai requisiti, abbiamo definito i nuovi casi d'uso per le operazioni di gestione dei suggerimenti da parte dell'utente.



2.6.3 Scenari Casi d'Uso

ID e Nome:	UC01: Richiesta suggerimento Base
Attore:	Utente registrato
Attore Secondario:	-
Descrizione:	L'utente tramite interfaccia UI può richiedere un suggerimento base a seconda della disponibilità
Pre-condizioni:	- Utente loggato. - Partita in corso con classe e difficoltà valide. - Sono disponibili suggerimenti per quella classe/difficoltà - L'utente ha suggerimenti disponibili

Sequenza di eventi principali:	1. L'Utente clicca un pulsante sulla UI, ad esempio: "Richiedi Suggerimento" 2. Il sistema verifica la disponibilità 3. Il sistema seleziona un suggerimento non ancora erogato nella sessione di gioco 4. Il suggerimento viene salvato e vengono aggiornati i contatori 5. Il sistema mostra a video il suggerimento mediante una finestra di pop-up 6. Il sistema permette all'utente di inserire il suggerimento come commento nella classe di test che sta scrivendo.
Post-condizioni:	- Un suggerimento base è stato erogato e salvato nello storico - Il contatore viene decrementato
Casi Correlati:	Richiesta Suggerimento; <include> visualizza Suggerimento; <extends>
Estensioni:	- Nessun suggerimento disponibile → 404 o risposta vuota - Tutti i suggerimenti sono stati erogati → risposta "Non ci sono altri suggerimenti" - Problemi di rete o time-out tra client e API Gateway.

ID e Nome:	UC02: Richiesta suggerimento avanzato
Attore primario:	Utente registrato
Attore secondario:	-
Descrizione:	L'utente già registrato e autenticato può richiedere un suggerimento avanzato, visualizzarlo e inserirlo nel codice come commento nella partita che sta effettuando.
Pre-Condizioni:	- Utente loggato. - Partita in corso con classe e difficoltà valide. - Sono disponibili suggerimenti per quella

	classe/difficoltà - L'utente ha suggerimenti disponibili - L'utente ha abbastanza crediti per richiedere il suggerimento
Sequenza di eventi principali:	1. Il caso d'uso inizia dopo che l'utente effettua i seguenti passi a. seleziona il menù a tendina 'Suggerimenti'; b. va nella sezione avanzati c. clicca sul pulsante 'Acquista suggerimento' 2. Il sistema controlla se possiede i crediti necessari per poter richiedere un suggerimento avanzato a. Se li possiede aggiorna i crediti residui 3. Il sistema controlla se ci sono suggerimenti disponibili a. Se ci sono aggiorna il numero di suggerimenti disponibili 4. Il sistema mostra a video il suggerimento mediante una finestra di pop-up, scelto in maniera randomica tra quelli disponibili e ancora non visualizzati. 5. Il sistema permette all'utente di inserire il suggerimento come commento nella classe di test che sta scrivendo.
Post-Condizioni:	- Il suggerimento è stato visualizzato e/o aggiunto al codice della classe di test come commento. - Un suggerimento avanzato è stato erogato e salvato nello storico - Il contatore viene decrementato - I crediti sono stati decrementati
Casi correlati:	Richiesta suggerimento; <<include>> Verifica crediti disponibili, Aggiorna numero crediti, Visualizza suggerimento, Inserisci suggerimento

Estensioni:	- Nessun suggerimento disponibile → 404 o risposta vuota - Tutti i suggerimenti sono stati erogati → risposta "Non ci sono altri suggerimenti" - Crediti non sufficienti - Problemi di rete o time-out tra client e API Gateway.
--------------------	---

ID e Nome:	UC03: Visualizza Storico Suggerimenti
Attore:	Utente registrato
Attore Secondario:	-
Descrizione:	L'utente può visualizzare una lista dei suggerimenti già richiesti nel corso della partita.
Pre-condizioni:	- Utente loggato. - Partita in corso con classe e difficoltà valide.
Sequenza di eventi principali:	1. L'utente seleziona "Storico suggerimenti". 2. Il sistema recupera i suggerimenti dell'utente ordinati (dal più recente). 3. Il sistema mostra la lista dei suggerimenti ricevuti
Post-condizioni:	- lo storico dei suggerimenti dell'utente è visualizzato correttamente a video
Casi Correlati:	-
Estensioni:	- Nessun suggerimento presente → Il sistema mostra un messaggio "Nessun suggerimento trovato".

ID e Nome:	UC05: Carica suggerimento
Attore:	Amministratore
Attore Secondario:	-
Descrizione:	L'amministratore può caricare un nuovo suggerimento nel DataBase
Pre-condizioni:	- Amministratore autenticato.
Sequenza di eventi principali:	1. Il caso d'uso inizia quando l'amministratore clicca su crea suggerimento 2. Il sistema legge i dati del suggerimento inseriti dall'amministratore 3. Il sistema crea un nuovo suggerimento con i dati dell'utente 4. Il sistema aggiorna il DataBase col nuovo suggerimento disponibile 5. Il sistema mostra un messaggio di conferma all'amministratore attraverso l'interfaccia grafica
Post-condizioni:	- Il nuovo suggerimento è salvato all'interno del DataBase ed è disponibile ai giocatori
Casi Correlati:	-
Estensioni:	- Se i campi className,difficulty e text del suggerimento sono vuoti viene mostrato un messaggio di errore all'amministratore attraverso l'interfaccia grafica

2.7 Analisi dell'Impatto

Si prevede la modifica dei seguenti moduli **T1**, **T5**, **T23**.

T1 – Il modulo T1 gestisce tutta la logica lato Amministratore. In particolare, esso si occupa del caricamento e salvataggio delle classi da testare.

Poiché sarà l'amministratore a caricare e salvare i suggerimenti, saranno necessarie delle modifiche: per l'implementazione del servizio richiesto e per l'ottenimento dal database dei suggerimenti disponibili. Dovranno essere implementati la Suggestion API, i controller dei suggerimenti, algoritmo di selezione dei suggerimenti, DB con la tabella suggerimenti;

T5 – Il modulo T5 si occupa di fornire i servizi necessari all'avvio della partita e dell'ambiente di editing, nonché l'interfaccia grafica e l'editor per la scrittura del codice. L'obiettivo del task è quello di fornire all'utente un meccanismo per la richiesta di un suggerimento; ciò deve essere fatto tramite interfaccia grafica, per cui è necessario apportare modifiche anche al codice del modulo T5. In particolare, si prevede di modificare l'interfaccia dell'editor aggiungendo opportuni widget come, ad esempio, uno o più pulsanti, che permettano la richiesta di suggerimenti, ed eventuali contatori.

T23 – Il modulo T23 gestisce tutte le procedure di registrazione e autenticazione degli utenti e, quindi anche del mantenimento dei dati di ogni utente attraverso un proprio database. Per ciò che concerne il task R3, per implementare il sistema dei crediti per i suggerimenti avanzati, si rende necessaria la modifica degli attributi dell'utente aggiungendo un contatore che mantenga il conteggio dei crediti che l'utente ha a disposizione, nonché l'introduzione di meccanismi per aggiornare il saldo crediti.

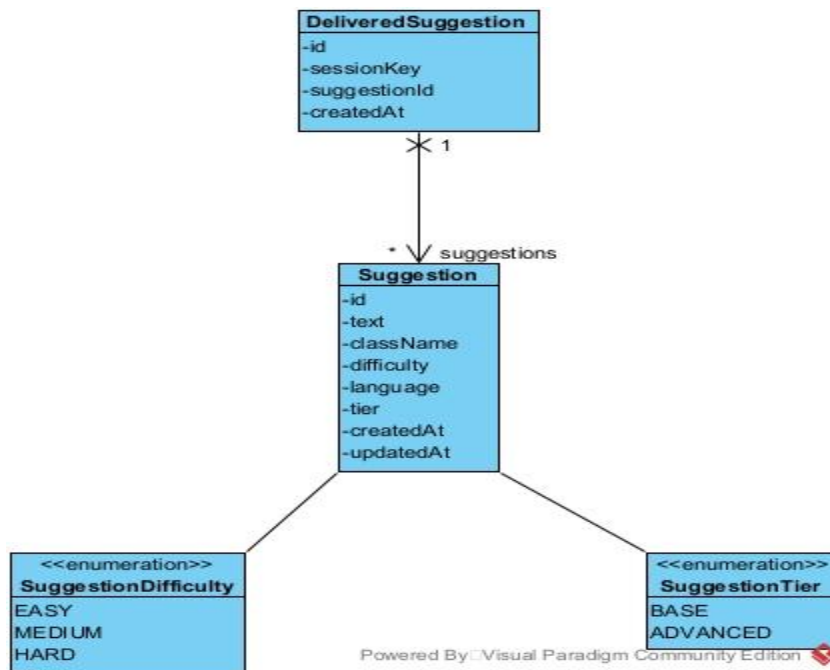
Modulo	Responsabilità Attuale	Descrizione della Modifica (Impatto)	Componenti Interessati
T1 (Admin)	Gestione logica Admin, caricamento/salvataggio classi.	Implementazione logica per gestione e recupero suggerimenti.	• Backend Controller (Suggerimenti) • Database (Nuova tabella Suggestions) • Algoritmo di selezione
T5 (Game UI)	Avvio partita, Editor di codice, GUI.	Aggiornamento GUI per permettere l'interazione utente con i suggerimenti.	• Widget Editor (Pulsante "Richiedi") • Visualizzatore Saldo Crediti • Gestione eventi UI
T23 (User)	Registrazione, Autenticazione, Dati Utente.	Estensione profilo utente per gestione valuta virtuale (crediti).	• Aggiornamento Schema Database • Logica di aggiornamento saldo

3. Progettazione della soluzione

3.1. Decisioni di Progetto

Per poter completare il task assegnato si è deciso di partire dal front-end gestito dal modulo T5, modificando l'interfaccia utente allo scopo di visualizzare il pulsante per la richiesta di suggerimenti durante la partita. Una volta implementato il lato front-

end in T5, è stato modificato il database in T1, quello che gestisce i dati dell'admin, in modo tale da fargli memorizzare anche i suggerimenti che l'amministratore inserisce nel gioco. Successivamente abbiamo modificato anche il database in T23, il cui compito è quello di conservare i dati degli utenti registrati, per associare a ogni profilo giocatore un contatore dei crediti a disposizione. Aggiornati i database e l'interfaccia utente si è passati al collegamento del lato front-end in T5 con il back-end in T1 e T23.



Per poter gestire la possibilità per un giocatore di richiedere un suggerimento, abbiamo ritenuto necessario aggiungere una classe model **Suggestion**, la quale rappresenta i dati relativi a un suggerimento. Essa ha un id univoco, una parte testuale che rappresenta il contenuto del suggerimento in questione, la classe per cui quel suggerimento è disponibile, la difficoltà alla quale il suggerimento potrà essere ricevuto, la lingua del suggerimento (tra le lingue supportate dall'applicazione), il grado del suggerimento (BASE/AVANZATO) e, infine, due timestamp per la creazione e l'ultimo aggiornamento del suggerimento.

La classe **Suggestion** è legata alla classe **DeliveredSuggestion**, la quale tiene traccia dei suggerimenti che sono già stati consegnati per quella sessione, in modo tale che lo stesso suggerimento non possa essere consegnato più volte nella stessa sessione di gioco.

Tra le due classi c'è una relazione di associazione di tipo 1:N, abbiamo preso questa decisione in quanto ogni suggerimento può essere consegnato molte volte, ma un

evento di consegna del suggerimento è unico all'interno della logica che abbiamo deciso di seguire.

3.2. Panoramica dei file modificati

Nel modulo T1 si è ritenuto necessario implementare la classe **SuggestionController**, la quale mappa tutte le rotte relative ai suggerimenti. In questa classe sono presenti gli endpoint, sia quelli utilizzati dai giocatori che quelli utilizzati dagli amministratori.

Gli endpoint per giocatori servono a:

- Richiedere un suggerimento base.
- Richiedere un suggerimento avanzato (Se il giocatore ha abbastanza crediti).
- Ottenere il numero di suggerimenti disponibili senza consumarne.

Gli endpoint per amministratori servono a:

- Inserire un nuovo suggerimento.
- Sostituire tutti i suggerimenti relativi a una determinata classe.
- Configurare il limite di suggerimenti base per difficoltà.
- Visualizzare la lista dei suggerimenti disponibili per una determinata coppia. classe-difficoltà.
- Eliminare un suggerimento dal Database.

In T23 è stata modificata la classe **PlayerProgressController** per permettere a essa di includere gli endpoint per la gestione dei crediti, tali endpoint si occupano di:

- Ottenere il numero di crediti che un giocatore ha a disposizione
- Aumentare il numero di crediti a disposizione quando un giocatore vince una partita.
- Diminuire il numero di crediti a disposizione quando un giocatore acquista. un suggerimento avanzato.

Anche il modulo service ha ricevuto delle modifiche, la creazione della classe **SuggestionService** permette di gestire tutta la business logic relativa a richiesta, aggiunta, eliminazione e modifica dei suggerimenti. Questa classe si occupa di:

- Costruire il contesto.
- Recuperare i suggerimenti disponibili dal database.
- Tenere traccia dei suggerimenti già mostrati nella sessione
- Selezionare casualmente un suggerimento da mostrare al giocatore

- Effettuare una chiamata REST a T23 per sottrarre i crediti al giocatore nel caso sia stato acquistato un suggerimento avanzato.
- Aggiungere suggerimenti al database.
- Eliminare suggerimenti dal database.
- Costruire il DTO di risposta contenente il suggerimento da mostrare.

Come era lecito pensare sono state aggiunte classi anche nel model (T1) e nel repository (T1), per rappresentare i dati dei suggerimenti presenti nel database.

Nel modulo model sono state aggiunte le classi:

- **Suggestion**
- **DeliveredSuggestion**
- Classi enum:
 - **SuggestionDifficulty**
 - **SuggestionTier**

Nel modulo repository sono state aggiunte le classi:

- **SuggestionRepository**
- **DeliveredSuggestionRepository.**

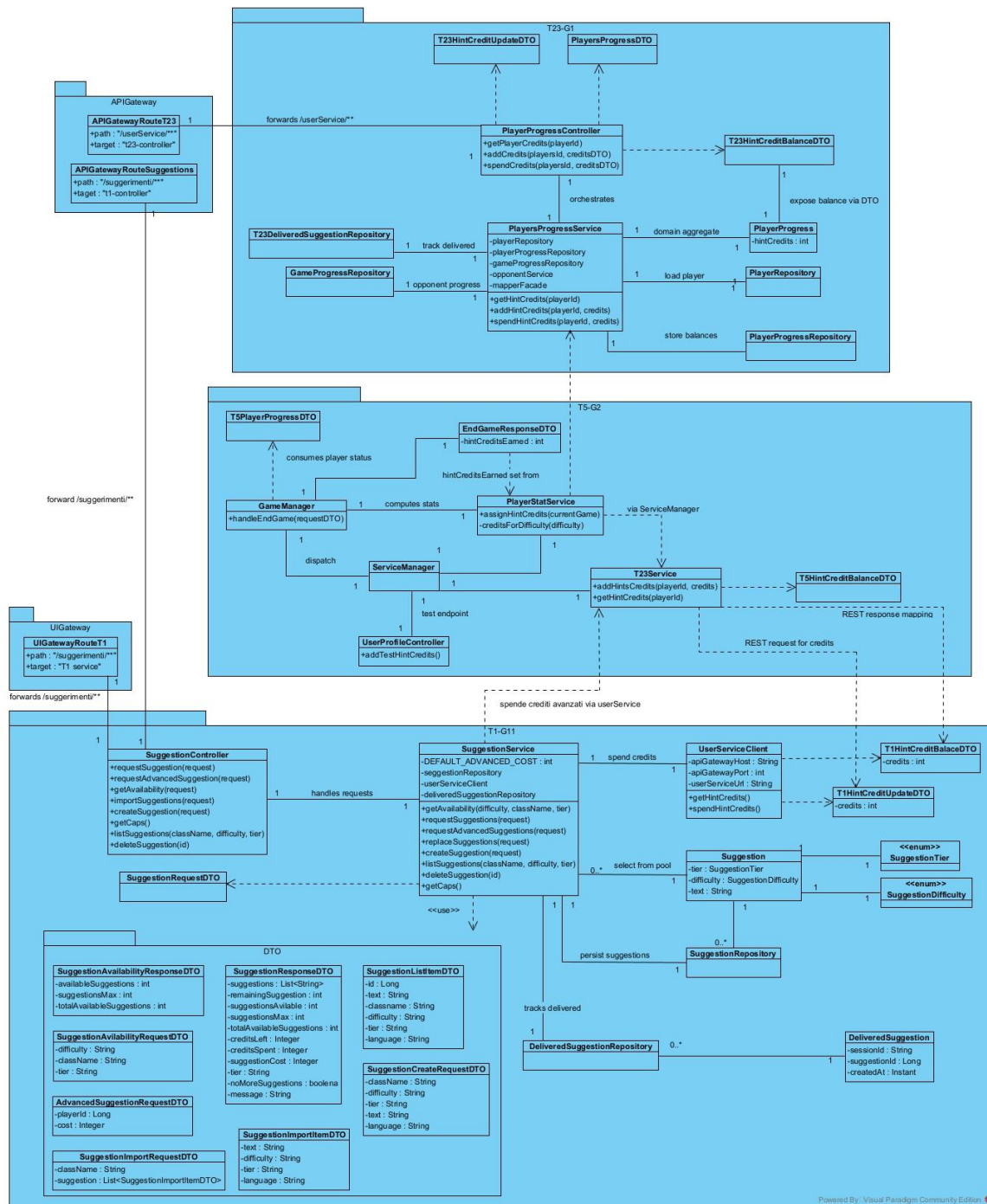
Classi DTO aggiunte:

- **AdvancedSuggestionRequestDTO:** payload per richiedere suggerimenti avanzati (include info giocatore/costo oltre alla richiesta base).
- **SuggestionRequestDTO:** payload per richiedere suggerimenti base (classe, difficoltà, partita, stato contatori).
- **SuggestionResponseDTO:** risposta con i suggerimenti restituiti e i metadati (rimasti, max, messaggi).
- **SuggestionAvailabilityRequestDTO:** payload per chiedere disponibilità/cap dei suggerimenti senza consumarli.
- **SuggestionAvailabilityResponseDTO:** risposta con disponibilità, massimo e storico già erogato.
- **SuggestionCreateRequestDTO:** payload per creare un nuovo suggerimento (testo, classe, difficoltà, tipo).
- **SuggestionListItemDTO:** elemento sintetico usato per liste di suggerimenti (id, testo, metadati).
- **SuggestionImportRequestDTO:** payload per import massivo di suggerimenti.
- **SuggestionImportItemDTO:** singolo record di import (classe/difficoltà/tipo/testo).

Nel T5 è stato aggiunto il file **Suggestion.js** per gestire tutta la logica frontend dei suggerimenti in partita: contatori (base/avanzati), richieste alle API, storici, e UI (modali/alert). Inoltre, sono stati modificati i file **editor.html**, per l'aggiunta dei pulsanti, e il file **Game_logic.js**, per configurare i suggerimenti quando viene avviata una partita.

3.3. Viste Architetture Aggiate

3.3.1. Class Diagram



In questo diagramma delle classi sono presenti solamente le classi aggiunte/modificate per richiedere suggerimenti base e avanzati.

In T1-G11 si gestisce la logica di selezione, i cap per difficoltà e la sessione/storico dei suggerimenti; in T23-G1 la gestione dei crediti del giocatore; in T5-G2 l'esperienza utente con feedback su contenuti e saldo. I gateway instradano le richieste verso i servizi appropriati.

Collaborazione tra strati:

- Controller: riceve le richieste, valida i parametri, delega.
- Service: applica regole (cap, sessione, selezione) e coordina la spesa crediti con il servizio utenti.
- Persistenza: mantiene catalogo suggerimenti e storico per sessione.
- Modelli/enum: descrivono difficoltà e categoria del suggerimento.
- DTO: incapsulano richieste/risposte, mantenendo l'API chiara e stabile.

La UI invoca */suggerimenti/*** (instradato dallo UI Gateway verso T1-G11) e */userService/*** (instradato dall'API Gateway verso T23-G1); la spesa dei crediti avviene come chiamata server-to-server T1-G11 → API Gateway → T23-G1.

Dopo un'operazione di import o pulizia del catalogo suggerimenti, il sistema rimuove automaticamente eventuali riferimenti non più validi dallo storico, evitando inconsistenze.

3.3.2. Sequence Diagrams

Diagramma per richiesta di suggerimento base

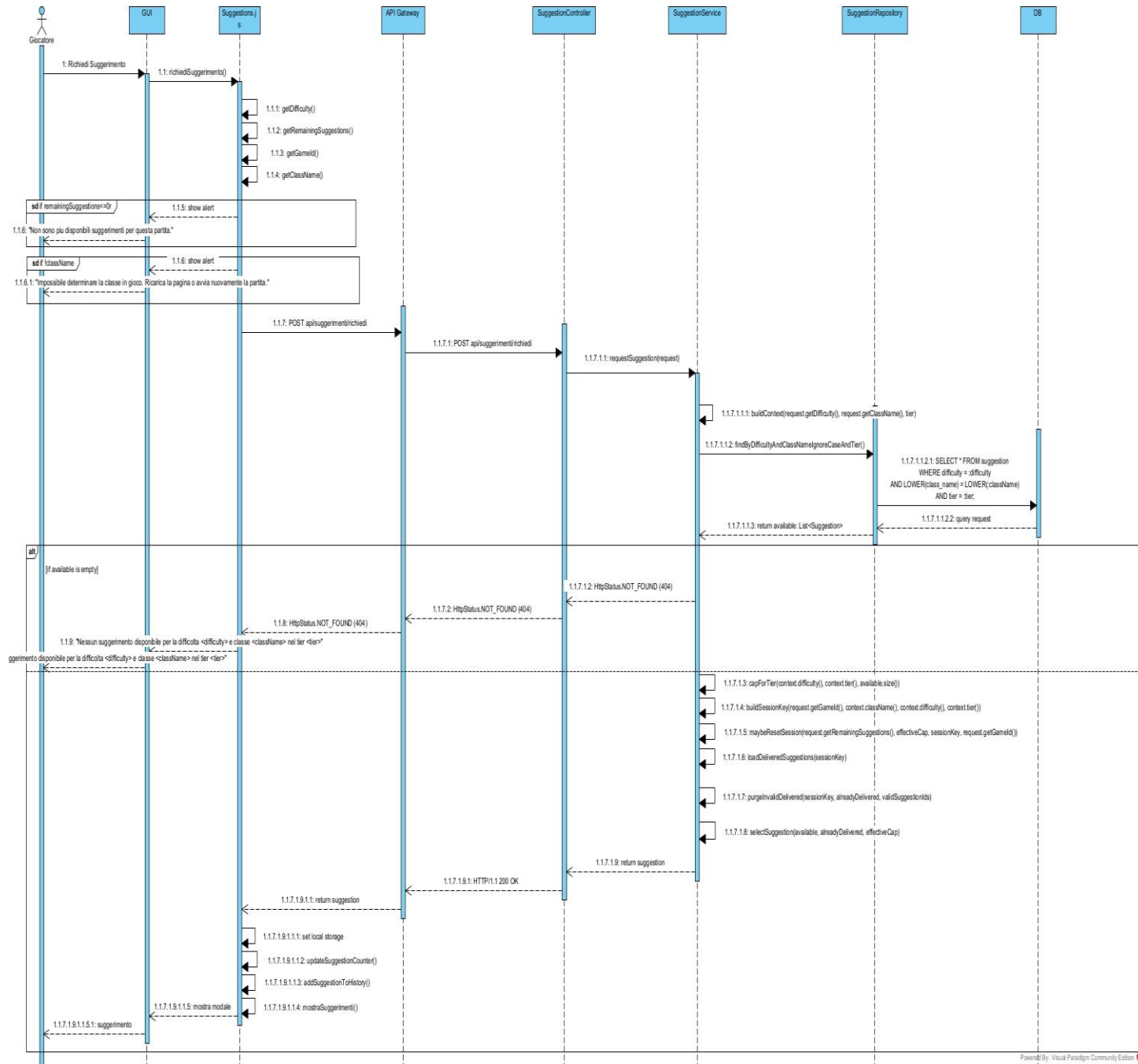


Diagramma per richiesta di suggerimento avanzato

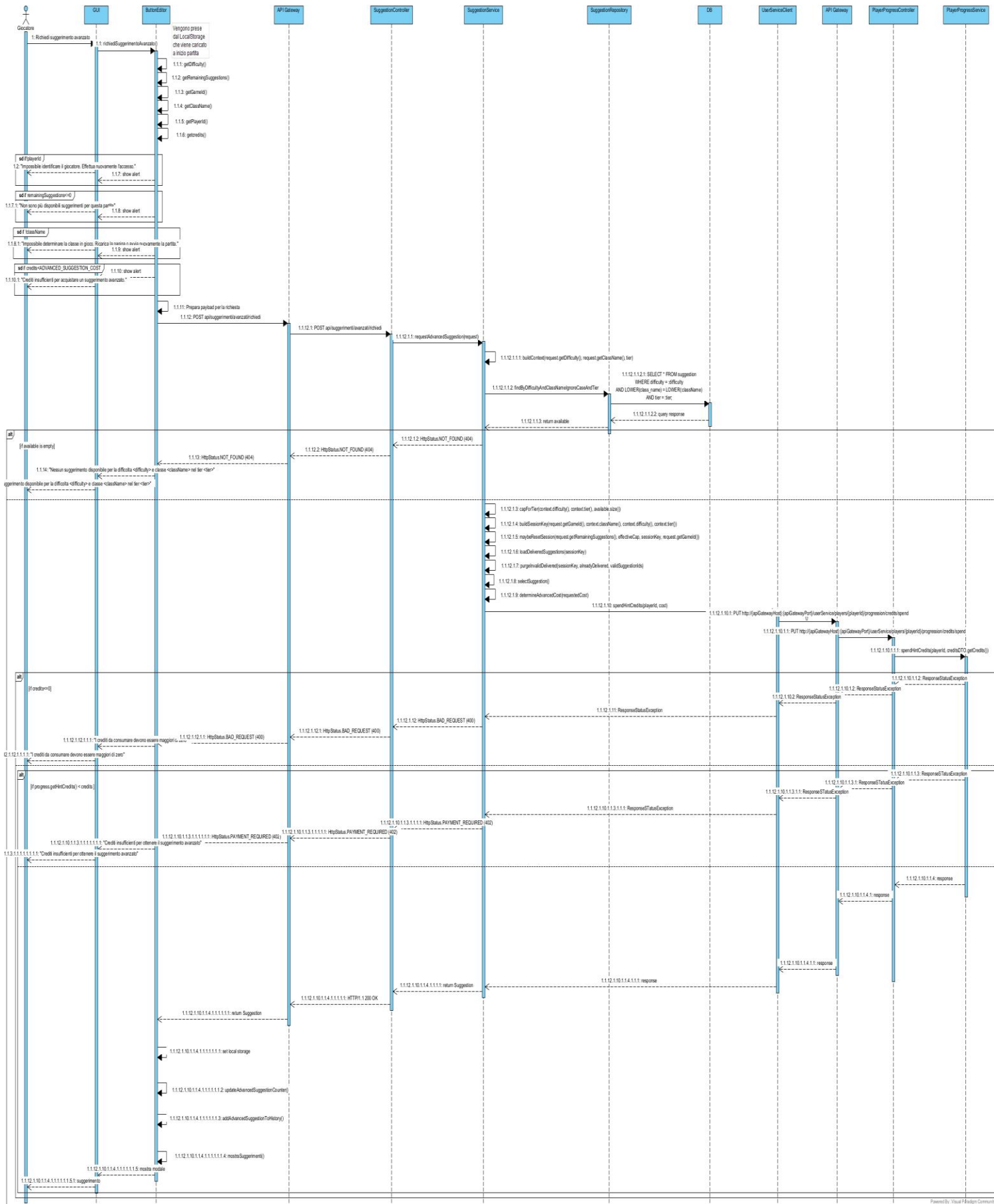


Diagramma per l'inizializzazione del contatore di suggerimenti

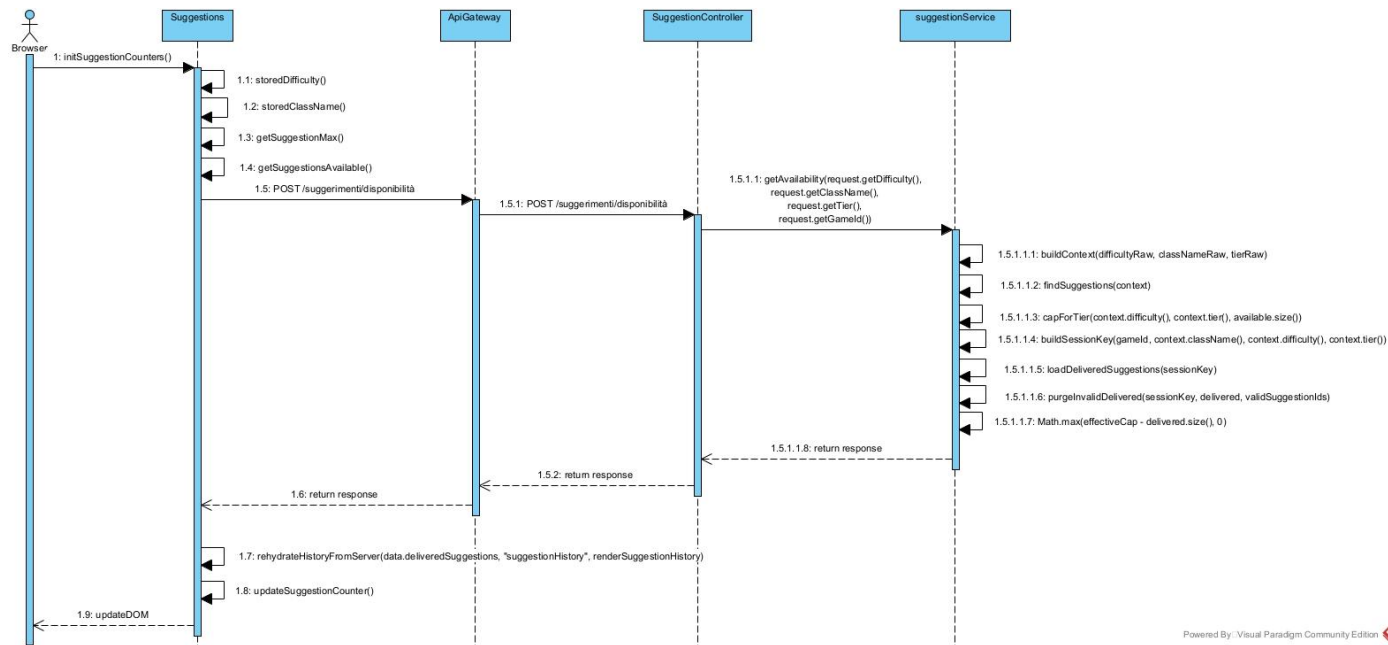
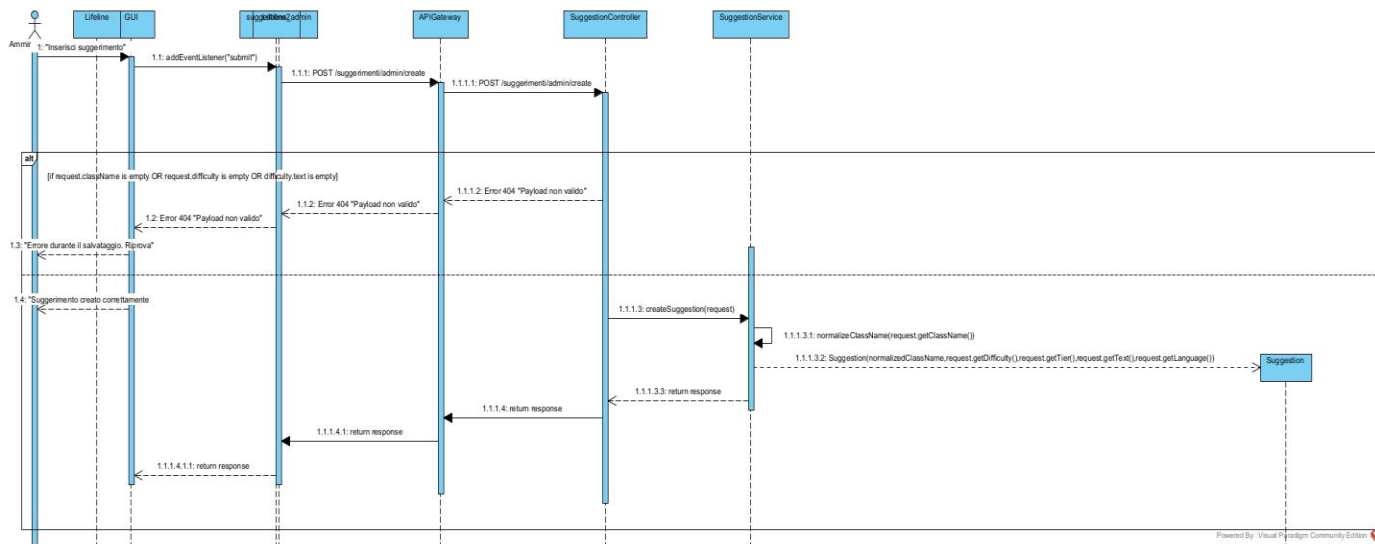
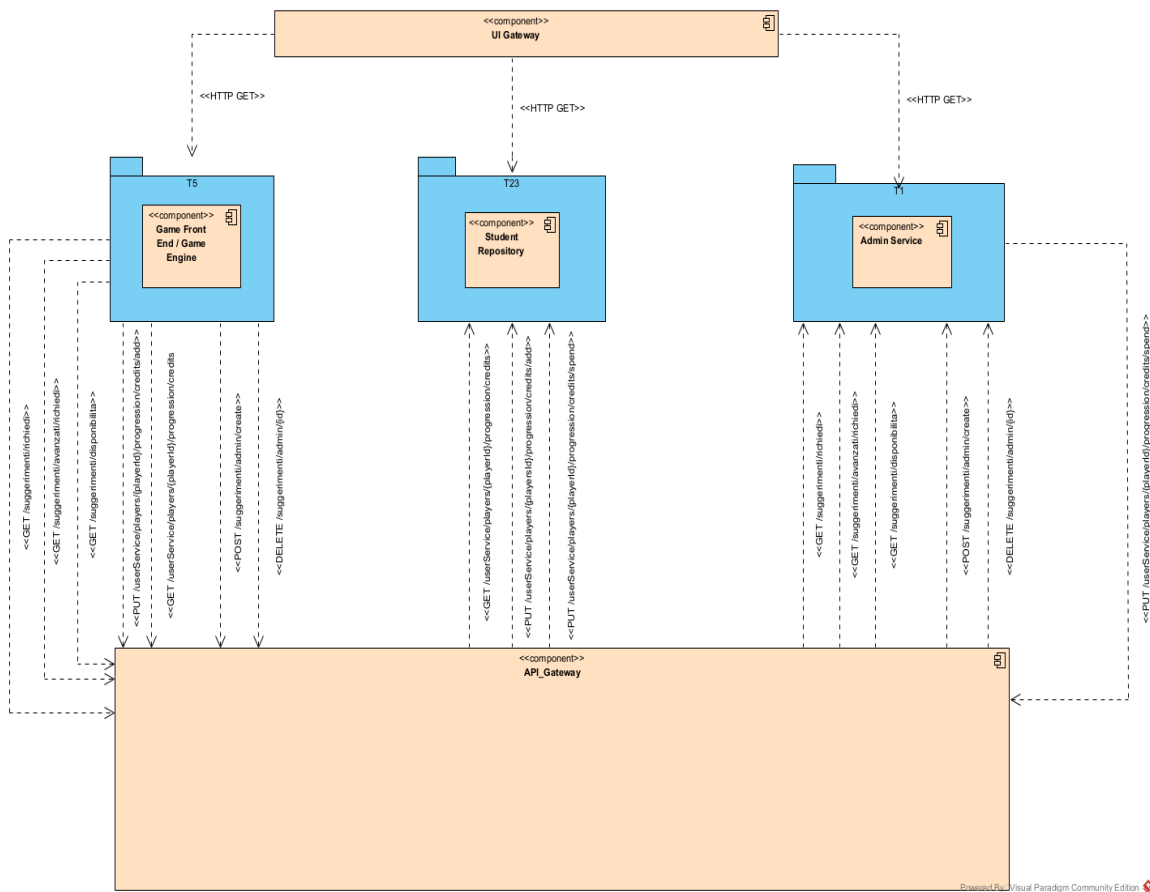


Diagramma per la creazione di un nuovo suggerimento



3.3.3. Component Diagram



Attraverso il Component Diagram mostrato in figura è possibile visualizzare le nuove rotte REST che si è ritenuto necessario implementare al fine di completare il task assegnato.

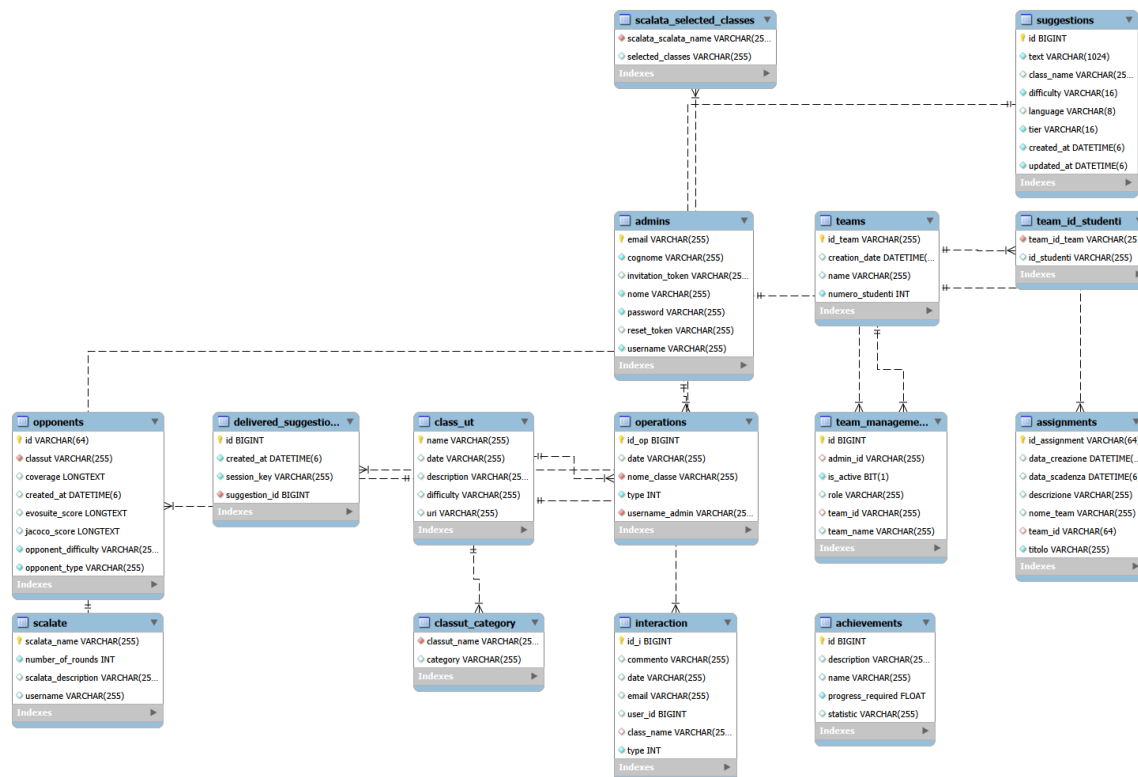
Nel dettaglio, queste sono state le rotte sviluppate:

- **GET /suggerimenti/riciedi**: restituisce un suggerimento base coerente con la classe sfidata e la difficoltà selezionata per quella partita;
- **GET /suggerimenti/avanzati/riciedi**: restituisce un suggerimento avanzato (ovvero a pagamento) se il giocatore ha abbastanza crediti;
- **GET /suggerimenti/disponibilita**: restituisce quanti suggerimenti sono ancora effettivamente disponibili per quella classe/difficoltà/tier;
- **POST /suggerimenti/admin/create**: crea un nuovo suggerimento (funzionalità utilizzabile dall'amministratore);
- **GET /suggerimenti/admin/list**: elenca all'amministratore i suggerimenti filtrati per classe/difficoltà/tier;
- **DELETE /suggerimenti/admin/{id}**: permette all'amministratore di eliminare un suggerimento specifico;

- **GET /userService/players/{playerId}/progression/credits:** restituisce il saldo crediti (utilizzabile per acquistare i suggerimenti avanzati) del giocatore;
- **PUT /userService/players/{playerId}/progression/credits/add:** aggiunge crediti al giocatore (es. dopo vittoria di una partita);
- **PUT /userService/players/{playerId}/progression/credits/spend:** scala crediti quando il giocatore acquista un suggerimento avanzato.

3.4. Modifiche ai Database

In precedenza, il servizio **T1** utilizzava MongoDB come sistema di persistenza dei dati. Si è poi scelto, coerentemente con la necessità di dover migrare ad un database relazionale e l'impossibilità di introdurre il nuovo dominio dei suggerimenti in un database che non permettesse la gestione di relazioni strutturate tra entità, di utilizzare MySQL.



Modello ER

Il database è stato poi esteso per ospitare il nuovo Sistema dei suggerimenti. È stata introdotta la tabella **suggestions**, che persiste ogni suggerimento mostrabile al giocatore, classe di riferimento, difficoltà, lingua e tier, oltre a timestamp di creazione/aggiornamento. A questa si affianca la tabella **delivered_suggestions**, pensata per evitare duplicati durante una partita, memorizzando l'identificativo della sessione (ovvero una chiave composta da gameId, classe, difficoltà e tier), l'id del suggerimento consegnato e il timestamp, imponendo pertanto l'unicità della coppia sessione-suggerimento.

I repository **SuggestionRepository** e **DeliveredSuggestionRepository** offrono Metodi CRUD per il tracciamento delle consegne, mentre il servizio **SuggestionService** usa queste tabelle per selezionare suggerimenti non ancora

serviti, applicare il limite massimo per difficoltà / tier e, per quelli avanzati, integrare il consumo di crediti tramite **UserServiceClient**.

3.5. Issue e Anti-pattern

- T1-G11: classe SuggestionService.java

src/.../com/groom/manvsclass/service/SuggestionService.java

☐ [Replace this usage of 'Stream.collect\(Collectors.toList\(\)\)' with 'Stream.toList\(\)' and ensure that the list is unmodified.](#) Intentionality

Maintainability Medium

Open Not assigned

L148 • 5min effort • 2 months ago

java16 +

☐ [Replace this usage of 'Stream.collect\(Collectors.toList\(\)\)' with 'Stream.toList\(\)' and ensure that the list is unmodified.](#) Intentionality

Maintainability Medium

Open Not assigned

L185 • 5min effort • 1 month ago

java16 +

In queste issue SonarQube ci dice di sostituire l'uso di 'Stream.collect(Collectors.toList())' con 'Stream.toList()' e assicurarci che la lista sia immutabile.

Il problema principale è che .collect(Collectors.toList()) restituisce in realtà una lista di tipo mutabile, mentre nella maggior parte dei casi si preferiscono liste non modificabili. Un nuovo collector per restituire una lista non modificabile è lo stream Stream.toList(). Questo tipo di issue presente molte volte nel codice è stata risolta proprio come suggeriva SonarQube.

☐ [Refactor this class declaration to use 'record SuggestionRequestContext\(String className, SuggestionDifficulty difficul...\)'.](#) Intentionality

Maintainability Medium

Open Not assigned

L397 • 20min effort • 4 days ago

java16 +

Rifattorizza questa dichiarazione di classe per utilizzare 'record SuggestionRequestContext(String className, SuggestionDifficulty difficul...)'.

I record rappresentano una struttura dati immutabile di sola lettura e dovrebbero essere utilizzati al posto della creazione di classi immutabili, poiché possono essere utilizzati in sicurezza nel codice di produzione. L'immutabilità dei record è garantita dal linguaggio Java stesso, mentre l'implementazione autonoma di classi immutabili

potrebbe causare alcuni bug e uno degli aspetti importanti dei record è che i campi final non possono essere sovrascritti.

Per risolverla la classe “private static final class SuggestionRequestContext { ... }” è stata sostituita con “private record SuggestionRequestContext(String className, SuggestionDifficulty difficulty, SuggestionTier tier) { ... }”

-
- T23-G1: non sono state riscontrate issue e Anti—pattern in questo modulo.
-

- T5-G2: classe T23Service.java

src/main/java/com/g2/interfaces/T23Service.java

☐ [Replace generic exceptions with specific library exceptions or a custom exception.](#) Intentionality

Maintainability Medium cwe error-handling ... +

Open Not assigned L226 • 20min effort • 6 months ago

Sostituire le eccezioni generiche con eccezioni di libreria specifiche o un'eccezione personalizzata. Le eccezioni generiche non dovrebbero mai essere generate, perché i consumatori sono costretti a intercettare eccezioni che non intendono gestire, che poi devono rilanciare. Abbiamo risolto con “catch (JsonProcessingException e)”.

☐ [Return an empty collection instead of null.](#) Intentionality

Maintainability Medium cert +

Open Not assigned L313 • 30min effort • 11 months ago

Restituisce una raccolta vuota anziché null, perché array e raccolte vuoti dovrebbero essere restituiti anziché null. Il tutto è stato risolto semplicemente con “return Collections.emptyList();”

☐ [Change this condition so that it does not always evaluate to "false"](#) Intentionality

Reliability Medium cwe symbolic-execution ... +

Open Not assigned L359 • 15min effort • 11 months ago

Modificare questa condizione in modo che non venga sempre valutata come "falsa", perché il codice eseguito in modo condizionale dovrebbe essere raggiungibile.

```
X if (response == null) {
V if (response == null || response.getBody() == null) {
    return new NotificationResponse();
X     } else {
X         return response.getBody();
    }
V     return response.getBody();
}
```

src/.../com/g2/game/gameDTO/EndGameDTO/EndGameResponseDTO.java

☐ [Rename this package name to match the regular expression `^\[a-z\_\]\(\.\[a-z\_\]\[a-z0-9\_\]\*\)\*\$`.](#)

Maintainability Low

Open Not assigned

Consistency convention L1 • 10min effort • 3 months ago

Rinomina questo nome del package in modo che corrisponda all'espressione regolare `^[a-z_](\.[a-z_][a-z0-9_]*)*$`.

Questo package che da errore è il seguente “package com.g2.game.gameDTO.EndGameDTO;”, per scelta di progettazione non abbiamo cambiato il nome del package.

src/main/java/com/g2/interfaces/T23Service.java

☐ [Remove this unnecessary cast to "Set".](#)

Maintainability Low

Open Not assigned

Intentionality redundant clumsy L250 • 5min effort • 6 months ago

Rimuovere questo cast non necessario su "Set" e non utilizzare cast ridondanti:

```
X return (Set<String>) callRestPut(endpoint, requestBody, null, null,
Set.class);
V return callRestPut(endpoint, requestBody, null, null, new
ParameterizedTypeReference<Set<String>>() {});
```

4. Implementazione e Struttura del Progetto

4.1. Moduli Aggiunti o Modificati

Modulo	Ruolo	Endpoint/Entry point principali	Note chiave
T1-G11 / manvclass	Suggetion API (base/avanzati) import/export, caps	/suggerimenti/** (SuggestionController)	Selezione suggerimenti, tracking consegnati, spesa crediti via userService
T23-G1 / userService	Credit API e progressi layer	/players/{id}/progression/** (PlayerProgressController)	Saldo/spesa crediti hint, exp, achievements, progressi per avversario
T5-G2 / t5 client	Frontend gameplay/UI	ServiceManager + T23Service/T1Service	Assegna crediti, exp, achievement client-side; invoca T1/T23
ui_gateway	Reverse proxy UI => T1	/suggerimenti/**	Forward verso T1
apiGateway	API gateway backend	/suggerimenti/** → T1, /userService/** → T23	Instradamento e policy di front-door

4.2. Refactoring e Modifiche

I. T1-G11 / manvclass:

- Endpoint: introdotto `SuggestionController` con `/suggerimenti/\*\*` che copre richieste base/avanzate, disponibilità, import e CRUD admin; i path sono pensati per essere stabili e facilmente integrabili da UI e gateway.
- Logica: `SuggestionService` centralizza l'algoritmo di selezione con caps per difficoltà/tier e traccia server-side i suggerimenti erogati per

evitare duplicati anche tra riavvii; per gli avanzati orchestra la spesa crediti via `UserServiceClient` passando dall'API Gateway`.

- c. Persistenza: definite le entity `Suggestion` e `DeliveredSuggestion`; i repository offrono filtri per className/difficulty/tier e operazioni di pulizia sugli import, mantenendo in sincronia tracking e catalogo.
- d. Routing: `ui\_gateway` e `apiGateway` sono allineati su `/suggerimenti/\*\*`, garantendo un front-door unico e semplificando la navigazione lato UI.

II. **T23-G1 / userService:**

- a. Controller: `PlayerProgressController` espone rotte per bilancio crediti (get/add/spend), progression per avversario e achievement globali, normalizzando le risposte in DTO consumabili dal client.
- b. Servizio: `PlayerProgressService` valida input e aggiorna l'esperienza, progressi e crediti in modo atomico; crea/aggiorna i `GameProgress` per avversario con flussi idempotenti dove possibile.
- c. Modello dati: `PlayerProgress` include `hintCredits` e collezioni di achievement; `GameProgressRepository` fornisce query parametriche su mode/class/type/difficulty per recuperi mirati.
- d. Gateway: tutte le API sono dietro `apiGateway` su `/userService/\*\*`, offrendo un path stabile e policy condivise per chiamate da T1/T5.

III. **T5-G2 / t5 client:**

- a. Gameplay: `PlayerStatService` assegna esperienza in base alla difficoltà, sblocca achievement di modalità/globali e attribuisce crediti suggerimenti usando `ServiceManager` come dispatcher verso T23.
- b. Fine partita: `GameManager` integra questi flussi alla chiusura turno/partita e valorizza `EndGameResponseDTO.hintCreditsEarned`;
- c. Integrazione: `T23Service` registra azioni REST tipizzate (progressi, exp, achievements, crediti); `UserProfileController` espone un endpoint di test per aggiungere crediti e facilitare verifiche.
- d. Configurazione: mapping servizi abilitati e path dei file di gamification/turno sono definiti in properties per tenere in sincronismo tra UI e backend.

IV. **ui_gateway:**

- a. Routing: la route dedicata inoltra `/suggerimenti/\*\*` verso T1, centralizzando header/policy del reverse proxy.

V. **apiGateway:**

- a. Instradamento: `suggestions-route` e `T23-route` convogliano rispettivamente verso T1 e T23 con path unificati, abilitando timeout/retry e monitoraggio consistenti lungo l'intera catena.

5. Test

5.1. Testing di Unità del microservizio T1

L'attività di collaudo del microservizio **T1** è stata progettata per garantire l'affidabilità delle funzionalità core e la stabilità delle interfacce esposte.

La metodologia adottata è rigorosamente quella dello **Unit Testing**. La scelta architetturale è stata quella di isolare completamente ogni componente dalle sue dipendenze esterne (database, altri microservizi), preferendo un approccio basato con **Mock**, rispetto all'Integration Testing. Questo ha permesso di verificare la logica di business e la gestione degli errori in modo deterministico e performante, simulando anche scenari limite difficilmente riproducibili in un ambiente integrato reale.

I test sono stati sviluppati utilizzando:

JUnit 5: Framework di base per l'esecuzione e l'asserzione dei test.

Mockito: Utilizzato estensivamente per creare i "Mock" delle dipendenze (SuggestionRepository, UserServiceClient), permettendo di definire comportamenti predeterminati essenziali per lo Unit Testing.

MockMvc: Utilizzato per simulare le chiamate HTTP verso i Controller in un ambiente controllato e privo di server reale.

5.1.1. Architettura dei test

La suite di test è strutturata su due livelli distinti, ciascuno testato in isolamento:

1. **Service Layer (SuggestionServiceTest.java):** Rappresenta il test di unità puro della logica di business.

La classe SuggestionService è testata disaccoppiandola completamente dai repository e dai client HTTP tramite l'uso di Mock. Non viene caricato alcun contesto Spring, garantendo che i test verifichino esclusivamente l'algoritmo interno e non l'infrastruttura.

- **Alcuni Scenari:** Vengono validati i calcoli dei CAP di difficoltà, la gestione transazionale dei crediti e la logica di generazione delle chiavi di sessione per i suggerimenti già consumati.
2. **Controller Layer(SuggestionControllerTest.java):** Rappresenta il test di unità dello strato Web.

Sebbene venga utilizzato il framework Spring MVC Test (@WebMvcTest), questo non costituisce un test di integrazione in senso classico. Il Service sottostante viene infatti sostituito da un @MockBean, impedendo la comunicazione con la logica di business reale.

Obiettivo: Questo approccio permette di verificare esclusivamente la correttezza della serializzazione JSON, la validazione formale degli input (DTO) e la mappatura degli stati HTTP (es. 402 Payment Required, 404 Not Found), senza coinvolgere la complessità del backend.

5.1.2. Unit Test Suggestion Service di T1

Nel seguito i test sono stati suddivisi in due tabelle:

- la prima fa riferimento ai test relativi alla richiesta dei suggerimenti da parte dell'utente
- la seconda fa riferimento ai test relativi alla gestione dei suggerimenti da parte dell'amministratore

Test funzioni utente						
ID	Descrizione	Precondizioni	Input	Postcondizioni	Risultato Atteso	Risultato
S01	Test richiesta suggerimenti base senza suggerimenti disponibili	Repository non contiene suggerimenti per la classe/difficoltà richiesta	SuggestionRequestDTO con className="MyClass", difficulty="EASY", remainingSuggestions=0	Nessun suggerimento salvato, nessun credito speso	Eccezione ResponseStatusException con status NOT_FOUND (404)	OK
S02	Test richiesta suggerimenti avanzati senza suggerimenti disponibili	Repository non contiene suggerimenti avanzati per la classe/difficoltà richiesta	AdvancedSuggestionRequestDTO con className="MyClass", difficulty="HARD", remainingSuggestions=0, playerId=7	Nessun suggerimento salvato, nessun credito speso	Response con noMoreSuggestions=true, suggestions vuota, tier="ADVANCED", tutti i contatori a 0	OK
S03	Test richiesta suggerimenti avanzati con successo e spesa crediti	Repository contiene suggerimenti avanzati, nessun suggerimento	AdvancedSuggestionRequestDTO con className="MyClass", difficulty="MEDIUM", playerId=7, cost=3	Crediti spesi (3), suggerimento persistito in DeliveredSuggestion con	Response con suggestions=["Usa un costruttore"], creditsLeft=5, creditsSpent=3, suggestionCost=3,	OK

		già consegnato nella sessione		sessionKey corretto	tier="ADVANCED"	
S04	Test getAvailability filtra suggerimenti consegnati non validi	Repository contiene 2 suggerimenti, DeliveredSuggestion contiene 1 ID valido e 1 ID invalido (99)	difficulty="EASY", className="MyClass", gameId=null, tier=null	Suggerimenti non validi eliminati dal DB	Response con availableSuggestions=1, suggestionsMax=2, totalAvailableSuggestions=2, deliveredSuggestions=["Suggerimento 1"], deleteBySessionKeyAndSuggestionIdIn chiamato con [99]	OK
S05	Test richiesta suggerimenti avanzati senza playerId	Repository contiene suggerimenti avanzati	AdvancedSuggestionRequestDTO senza playerId (null)	Nessun credito speso, nessun suggerimento salvato	Eccezione ResponseStatusException con status BAD_REQUEST (400)	OK
S06	Test reset sessione quando il contatore client si resetta	Repository contiene suggerimenti, remainingSuggestions > 0 indica reset	SuggestionRequestDTO con remainingSuggestions=1	Sessione resettata (deleteBySessionKey chiamato), nuovo suggerimento salvato	Response con suggestions=["Suggerimento"], deliveredSuggestionRepository.deleteBySessionKey chiamato	OK
S11	Test getCaps restituisce valori configurati	Nessuna	Nessun input	Nessuna modifica al DB	Map con EASY=10, MEDIUM=5, HARD=2	OK
S12	Test richiesta suggerimenti con difficoltà invalida	Nessuna	SuggestionRequestDTO con difficulty="IMPOSSIBLE"	Nessun suggerimento processato	Eccezione ResponseStatusException con status BAD_REQUEST (400)	OK
S18	Test richiesta suggerimenti con gameId valido NON resettata sessione	Repository contiene suggerimenti	SuggestionRequestDTO con className="MyClass", difficulty="EASY", remainingSuggestions=10, gameId=42	Suggerimento consegnato, sessione NON resettata anche con remainingSuggestions alto	Response con suggestions=["Suggerimento"], sessionKey="game-42"	OK
S19	Test getAvailability con gameId usa sessionKey corretto	Repository contiene 1 suggerimento EASY/BASE	difficulty="EASY", className="MyClass", tier=null, gameId=99	Nessuna modifica	Response con availableSuggestions=1, findBySessionKey chiamato con "game-99"	OK
S20	Test richiesta	Repository	SuggestionRequestDTO	1 suggerimento	Response con	OK

	suggerimenti EASY limitata a cap 10	contiene 15 suggerimenti EASY/BASE per MyClass	con className="MyClass", difficulty="EASY", remainingSuggestions=0	consegnato	suggestionsMax=10, totalAvailableSuggestions=10, remainingSuggestions=9 (NON 14)	
S21	Test richiesta suggerimenti MEDIUM limitata a cap 5	Repository contiene 10 suggerimenti MEDIUM/BASE per MyClass	SuggestionRequestDTO con className="MyClass", difficulty="MEDIUM", remainingSuggestions=0	1 suggerimento consegnato	Response con suggestionsMax=5, totalAvailableSuggestions=5, remainingSuggestions=4 (NON 9)	OK
S22	Test richiesta suggerimenti HARD limitata a cap 2	Repository contiene 5 suggerimenti HARD/BASE per MyClass	SuggestionRequestDTO con className="MyClass", difficulty="HARD", remainingSuggestions=0	1 suggerimento consegnato	Response con suggestionsMax=2, totalAvailableSuggestions=2, remainingSuggestions=1 (NON 4)	OK
S23	Test suggerimenti avanzati SENZA cap su numero disponibili	Repository contiene 20 suggerimenti EASY/ADVANCED per MyClass	AdvancedSuggestionRequestDTO con className="MyClass", difficulty="EASY", remainingSuggestions=0, playerId=7, cost=2	1 suggerimento consegnato, 2 crediti spesi	Response con suggestionsMax=20, totalAvailableSuggestions=20, remainingSuggestions=19 (tutti disponibili, NO limite dei 10 di EASY)	OK
S24	Test raggiungimento cap imposta noMoreSuggestions a true	Repository contiene 2 suggerimenti HARD/BASE, 1 già consegnato	SuggestionRequestDTO con className="MyClass", difficulty="HARD", remainingSuggestions=0	2° suggerimento consegnato, cap raggiunto (2/2)	Response con suggestions.size=1, remainingSuggestions=0, noMoreSuggestions=true	OK
S25	Test getAvailability con tutti i suggerimenti già consegnati	Repository contiene 2 suggerimenti HARD/BASE, entrambi già consegnati	difficulty="HARD", className="MyClass", tier=null, gameId=null	Nessuna modifica	Response con availableSuggestions=0, suggestionsMax=2, deliveredSuggestions.size=2	OK

Test funzioni amministratore

ID	Descrizione	Precondizioni	Input	Postcondizioni	Risultato Atteso	Risultato
S07	Test creazione suggerimento con testo vuoto	Nessuna	SuggestionCreateRequest DTO con text=" " (solo spazi)	Nessun suggerimento creato	Eccezione ResponseStatusException con status BAD_REQUEST (400)	OK
S08	Test eliminazione suggerimento non esistente	Repository non contiene suggerimento con id=42	suggestionId=42	Nessun suggerimento eliminato	Eccezione ResponseStatusException con status NOT_FOUND (404)	OK
S09	Test listSuggestions con tier specificato	Repository contiene suggerimenti con tier=BASE	className="MyClass", difficulty="EASY", tier="BASE"	Nessuna modifica al DB	Lista con 1 elemento: SuggestionListItemDTO con tier="BASE", language="it"	OK
S10	Test listSuggestions senza tier specificato (fallback)	Repository contiene suggerimenti con tier=null	className="MyClass", difficulty="MEDIUM", tier=null	Nessuna modifica al DB	Lista con 1 elemento: SuggestionListItemDTO con tier="BASE" (valore di fallback)	OK
S13	Test listSuggestions con tier invalido	Nessuna	className="MyClass", difficulty="EASY", tier="GOLD"	Nessuna query al DB	Eccezione ResponseStatusException con status BAD_REQUEST (400)	OK
S14	Test creazione suggerimento con className vuoto	Nessuna	SuggestionCreateRequest DTO con className=" "	Nessun suggerimento creato	Eccezione ResponseStatusException con status BAD_REQUEST (400)	OK
S15	Test creazione suggerimento con successo e normalizzazione testo	Nessuna	SuggestionCreateRequest DTO con className="MyClass", difficulty="MEDIUM", text="Suggerimento con spazi ", tier="BASE", language="en"	Suggerimento salvato con testo normalizzato	Suggestion con text="Suggerimento con spazi" (senza spazi iniziali/finali), className="MyClass", difficulty=MEDIUM, tier=BASE, language="en"	OK
S16	Test creazione suggerimento con language null usa default italiano	Nessuna	SuggestionCreateRequest DTO con className="MyClass", difficulty="EASY", text="Suggerimento", tier="BASE",	Suggerimento salvato con language="it"	Suggestion con language="it" (valore di default)	OK

			language=null			
S17	Test replaceSuggestions elimina vecchi e salva nuovi	Repository contiene vecchi suggerimenti per TestClass	SuggestionImportRequest DTO con className="TestClass", suggestions=[{difficulty:"EASY", text:"Primo suggerimento", tier:"BASE"}, {difficulty:"HARD", text:"Secondo suggerimento", tier:"ADVANCED"}]	Vecchi suggerimenti eliminati, 2 nuovi suggerimenti salvati	deleteByClassNameIgnore Case chiamato con "TestClass", saveAll chiamato con lista di 2 Suggestion (EASY/BASE e HARD/ADVANCED)	OK

5.1.3. Unit Test Suggestion Controller in T1

In questa sezione sono elencati tutti i test eseguiti sulle rotte REST del SuggestionController.

ID	Descrizione	Precondizioni	Input	Postcondizioni	Risultato Atteso	Risultato
C01	Test endpoint POST /suggerimenti/riciedi	Service mockato restituisce response valida	POST con body: {className:"MyClass", difficulty:"EASY", remainingSuggestions:1}	Nessuna modifica	HTTP 200, JSON con suggestions=["Suggerimento"], tier="BASE"	OK
C02	Test endpoint POST /suggerimenti/avanzati/riciedi	Service mockato restituisce response con crediti	POST con body: {className:"MyClass", difficulty:"HARD", remainingSuggestions:0, playerId:9, cost:2}	Nessuna modifica	HTTP 200, JSON con suggestions=["Suggerimento avanzato"], tier="ADVANCED", creditsLeft=3, creditsSpent=2, suggestionCost=2	OK
C03	Test endpoint POST /suggerimenti/disponibilita	Service mockato restituisce disponibilità	POST con body: {className:"MyClass", difficulty:"EASY", tier:"BASE", gameId:1}	Nessuna modifica	HTTP 200, JSON con availableSuggestions=2, suggestionsMax=3, deliveredSuggestions=["Suggerimento 1"]	OK
C04	Test endpoint POST /suggerimenti/import	Service mockato esegue replaceSuggestions	POST con body: {className:"MyClass", suggestions:[{difficulty:"EASY", text:"Suggerimento", tier:"BASE"}]}	Suggerimenti sostituiti	HTTP 204 No Content	OK

C05	Test endpoint POST /suggerimenti/admin/create	Service mockato crea suggerimento	POST con body: {className:"MyClass", difficulty:"MEDIUM", text:"Suggerimento", tier:"BASE", language:"it"}	Suggerimento creato	HTTP 201 Created	OK
C06	Test endpoint POST /suggerimenti/config (getCaps)	Service mockato restituisce configurazione	POST senza body	Nessuna modifica	HTTP 200, JSON con EASY=10, MEDIUM=5, HARD=2	OK
C07	Test endpoint GET /suggerimenti/admin/list	Service mockato restituisce lista	GET con params: className=MyClass, difficulty=EASY, tier=BASE	Nessuna modifica	HTTP 200, JSON array con 1 elemento: {className:"MyClass", tier:"BASE"}	OK
C08	Test endpoint DELETE /suggerimenti/admin/{id}	Service mockato elimina suggerimento	DELETE /suggerimenti/admin/7	Suggerimento eliminato	HTTP 204 No Content	OK
C09	Test validazione POST /suggerimenti/riciedi con campi mancanti	Nessuna	POST con body: {} (vuoto)	Service NON invocato	HTTP 400 Bad Request	OK
C10	Test validazione POST /suggerimenti/avanzati/riciedi senza playerId	Nessuna	POST con body senza playerId	Service NON invocato	HTTP 400 Bad Request	OK
C11	Test validazione POST /suggerimenti/disponibilita senza difficulty	Nessuna	POST con body: {className:"MyClass"} (senza difficulty)	Service NON invocato	HTTP 400 Bad Request	OK
C12	Test validazione POST /suggerimenti/import con payload vuoto	Nessuna	POST con body: {className:"MyClass", suggestions:[]} (lista vuota)	Service NON invocato	HTTP 400 Bad Request	OK
C13	Test validazione GET	Nessuna	GET con solo param className=MyClass	Service NON	HTTP 400 Bad Request	OK

	/suggerimenti/admin/list senza parametri richiesti		(manca difficulty)	invocato		
C14	Test propagazione errore NOT_FOUND dal service	Service lancia ResponseStatusException NOT_FOUND	POST /suggerimenti/richiedi con dati validi	Nessuna modifica	HTTP 404 Not Found	OK
C15	Test propagazione errore PAYMENT_REQUIRED per crediti insufficienti	Service lancia ResponseStatusException PAYMENT_REQUIRED	POST /suggerimenti/avanzati/richiedi con dati validi	Nessuna modifica	HTTP 402 Payment Required	OK
C16	Test validazione GET /suggerimenti/admin/list senza className	Nessuna	GET con solo param difficulty=EASY (manca className)	Service NON invocato	HTTP 400 Bad Request	OK
C17	Test propagazione errore BAD_GATEWAY dal service	Service lancia ResponseStatusException BAD_GATEWAY	POST /suggerimenti/config	Nessuna modifica	HTTP 502 Bad Gateway	OK
C18	Test requestSuggestion verifica tutti i campi della response	Service restituisce response completa	POST /suggerimenti/richiedi con body: {className:"MyClass", difficulty:"EASY", remainingSuggestions:2}	Nessuna modifica	HTTP 200, JSON con suggestions.size=2, remainingSuggestions=3, suggestionsAvailable=3, suggestionsMax=10, totalAvailableSuggestions=10, noMoreSuggestions=false, tier="BASE"	OK
C19	Test requestAdvancedSuggestion verifica campi crediti	Service restituisce response con crediti	POST /suggerimenti/avanzati/richiedi con body: {className:"MyClass", difficulty:"HARD", remainingSuggestions:0, playerId:5, cost:3}	Nessuna modifica	HTTP 200, JSON con creditsLeft=7, creditsSpent=3, suggestionCost=3, tier="ADVANCED", noMoreSuggestions=true	OK
C20	Test getAvailability verifica array deliveredSuggestions	Service restituisce disponibilità con 2 suggerimenti consegnati	POST /suggerimenti/disponibilita con body: {className:"MyClass", difficulty:"MEDIUM", tier:"BASE", gameId:1}	Nessuna modifica	HTTP 200, JSON con availableSuggestions=3, suggestionsMax=5, totalAvailableSuggestions=5, deliveredSuggestions=["Suggerimento 1", "Suggerimento 2"]	OK

C21	Test createSuggestion con className null	Nessuna	POST /suggerimenti/admin/create con body: {difficulty:"EASY", text:"Suggerimento", tier:"BASE"} (className omissa)	Service NON invocato	HTTP 400 Bad Request (validazione fallita)	OK
C22	Test createSuggestion con text null	Nessuna	POST /suggerimenti/admin/create con body: {className:"MyClass", difficulty:"EASY", tier:"BASE"} (text omissa)	Service NON invocato	HTTP 400 Bad Request (validazione fallita)	OK
C23	Test deleteSuggestion con id zero	Service processa la richiesta con id=0	DELETE /suggerimenti/admin/0	Suggerimento con id=0 eliminato	HTTP 204 No Content	OK
C24	Test listSuggestions con tier invalido propaga errore	Service lancia ResponseStatusException BAD_REQUEST	GET /suggerimenti/admin/list con params: className=MyClass, difficulty=EASY, tier=INVALID	Nessuna modifica	HTTP 400 Bad Request	OK
C25	Test requestSuggestion verifica parametri esatti passati al service	Service restituisce response valida	POST /suggerimenti/richiedi con body: {className:"TestClass", difficulty:"MEDIUM", remainingSuggestions:5, gameId:42}	Nessuna modifica	Service invocato con parametri esatti, times(1)	OK
C26	Test getAvailability verifica parametri esatti passati al service	Service restituisce disponibilità	POST /suggerimenti/disponibilita con body: {className:"VerifyClass", difficulty:"HARD", tier:"ADVANCED", gameId:99}	Nessuna modifica	Service.getAvailability invocato con parametri ("HARD", "VerifyClass", "ADVANCED", 99L), times(1)	OK
C27	Test importSuggestions con lista molto grande (50 elementi)	Nessuna	POST /suggerimenti/import con body: {className:"LargeClass", suggestions:[50 elementi]}	50 suggerimenti importati	HTTP 204 No Content, service.replaceSuggestions invocato con lista di 50 elementi	OK
C28	Test deleteSuggestion verifica ID esatto passato al service	Service elimina suggerimento con id=123	DELETE /suggerimenti/admin/123	Suggerimento con id=123 eliminato	Service.deleteSuggestion invocato con id=123, mai con 122 o 124	OK
C29	Test listSuggestions verifica tutti i parametri e campi	Service restituisce lista con 2 elementi	GET /suggerimenti/admin/list con params: className=TargetClass,	Nessuna modifica	HTTP 200, JSON array con 2 elementi, tutti i campi verificati (className, difficulty, tier, language)	OK

	response		difficulty=MEDIUM, tier=ADVANCED			
C30	Test createSuggestion success restituisce 201 Created	Service crea suggerimento	POST /suggerimenti/admin/cre ate con body: {className:"NewClass", difficulty:"HARD", text:"Nuovo suggerimento", tier:"BASE", language:"it"}	Suggerimento creato con id=100	HTTP 201 Created, service invocato 1 volta	OK
C31	Test getCaps verifica mappa completa restituita	Service restituisce configurazione caps	POST /suggerimenti/config	Nessuna modifica	HTTP 200, JSON con EASY=10, MEDIUM=5, HARD=2, dimensione mappa=3	OK

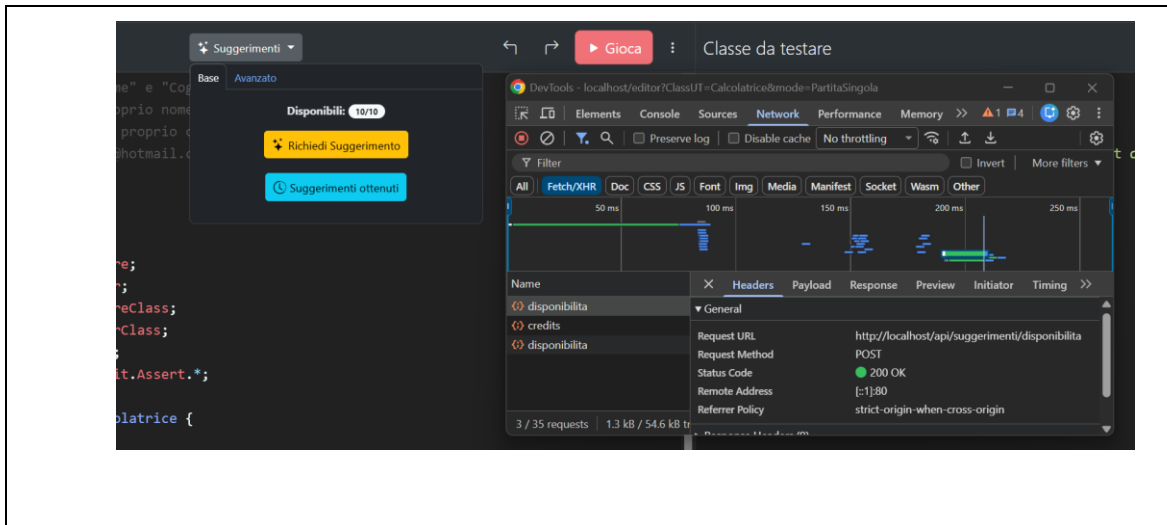
5.2. Test di Sistema

A completamento dei test di unità, è stato definito un piano di **Test di Sistema** per validare l'integrazione tra il frontend (T5), il backend (T1) e il servizio crediti (T23). Questi test verificano i flussi utente completi in un ambiente simile alla produzione.

I test sono stati effettuati in un ambiente in cui tutti i moduli erano attivi con delle classi precaricate e che l'utente sia correttamente autenticato con crediti iniziali ≥ 2 .

I casi coprono flussi base e avanzati, disponibilità, contatori, deduzione crediti, consistenza su refresh e funzionalità admin.

TS-SUG-01 — Disponibilità suggerimenti base (cap e conteggio)
Precondizioni: suggerimenti base presenti; partita attiva con classe e difficoltà note.
Passi: 1. Avvia una partita in T5 con classe e difficoltà note. 2. Apri il tab "Suggerimenti". 3. Attendi il caricamento della disponibilità (chiamata /api/suggerimenti/disponibilita).
Risultato atteso: • Il contatore in UI mostra un max coerente con il cap per difficoltà (EASY 10, MEDIUM 5, HARD 2) o con il numero reale disponibile, se inferiore. • Nessun suggerimento viene consumato.



TS-SUG-02 — Richiesta suggerimento base

Precondizioni:

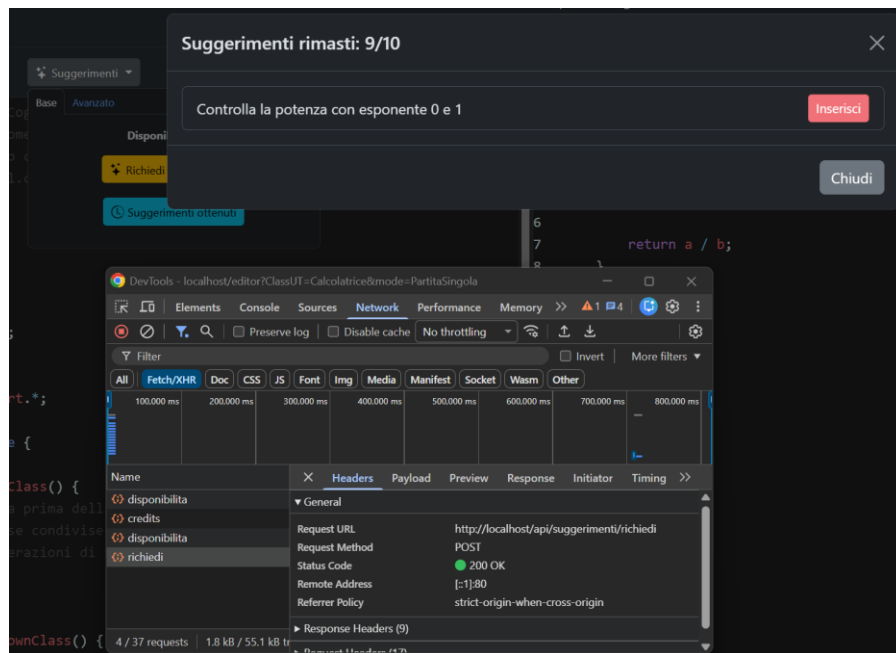
- Disponibilità > 0.
- Partita attiva con classe e difficoltà note.

Passi:

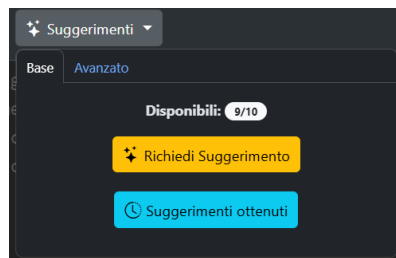
1. Clicca “Richiedi suggerimento”.
2. Verifica la modale con il testo del suggerimento.

Risultato atteso:

- Il backend risponde con un suggerimento non vuoto e viene mostrato a video.



- Il contatore in UI si decrementa.



TS-SUG-03 — Esaurimento suggerimenti base

Precondizioni: numero di richieste pari al cap (o al numero reale disponibile).

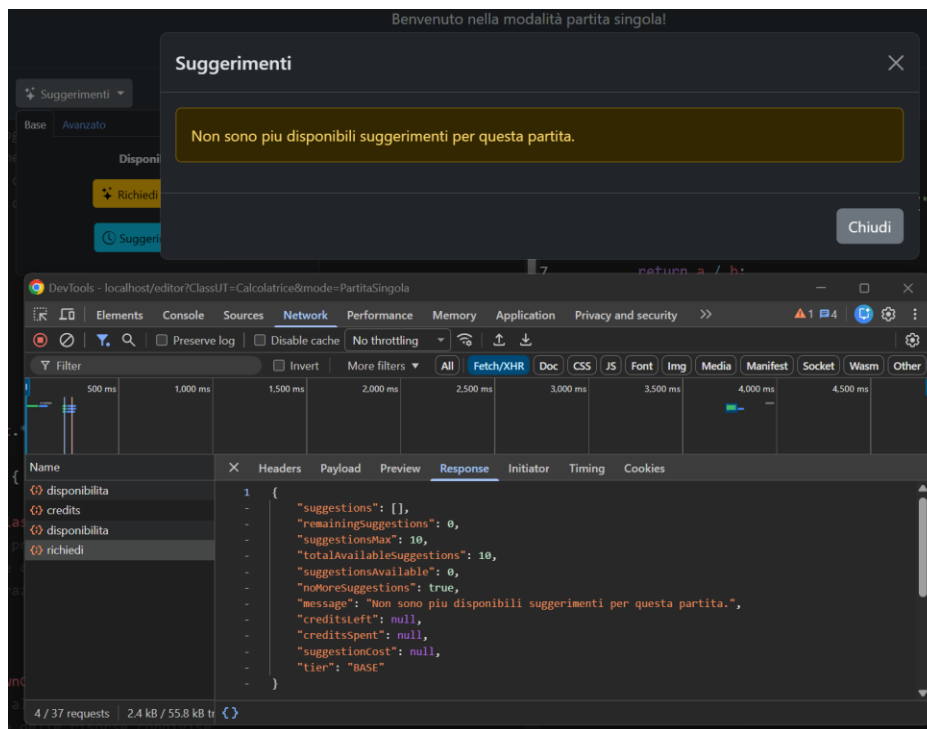
Passi:

1. Richiedi suggerimenti fino a esaurimento.
2. Richiedi ancora un suggerimento.

Risultato atteso:



- Il backend restituisce noMoreSuggestions=true e messaggio “Non sono piu disponibili suggerimenti per questa partita.”
- La UI mostra un alert/modale con lo stesso messaggio.



TS-SUG-04 — Suggerimenti base non ripetuti in partita

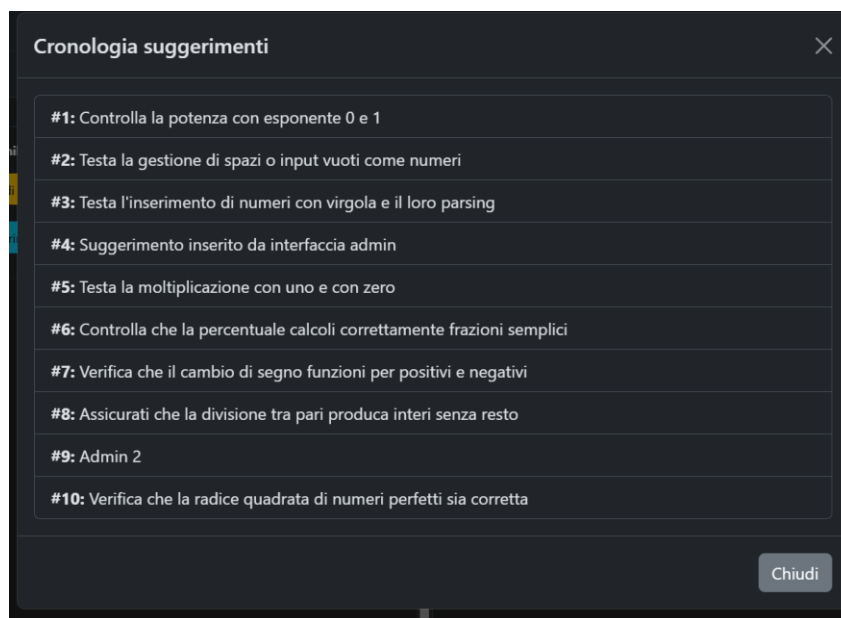
Precondizioni: almeno 2 suggerimenti base disponibili.

Passi:

1. Richiedi suggerimenti base ripetutamente.
2. Osserva la lista restituita.

Risultato atteso:

- Non vengono ripetuti suggerimenti già consegnati finché ce ne sono di nuovi.



TS-SUG-05 — Disponibilità suggerimenti avanzati

Precondizioni: suggerimenti avanzati presenti; utente autenticato.

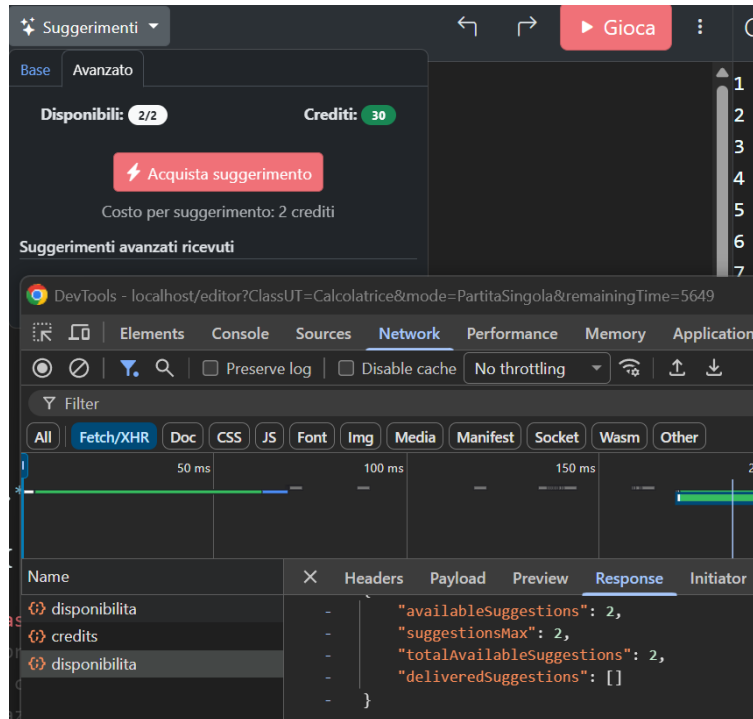
Passi:

1. Apri tab “Suggerimenti avanzati”.
2. Attendi la disponibilità

(chiamata /api/suggerimenti/disponibilita con tier=ADVANCED).

Risultato atteso:

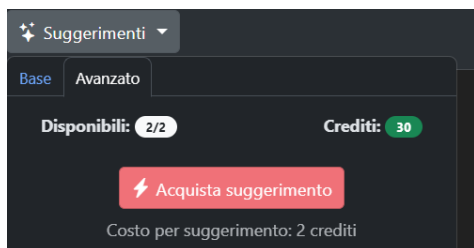
- Contatore avanzati valorizzato correttamente.
- Nessun suggerimento avanzato consumato.



TS-SUG-06 — Richiesta suggerimento avanzato con crediti sufficienti

Precondizioni: crediti utente ≥ 2 ; disponibilità avanzati > 0 .

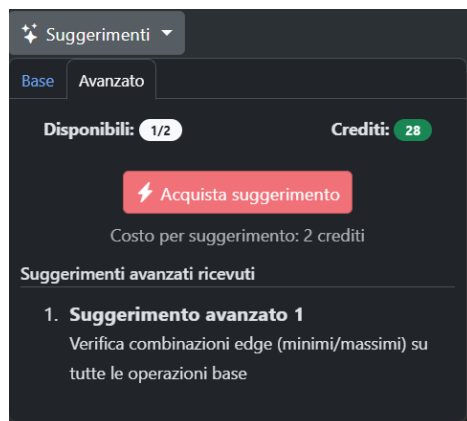
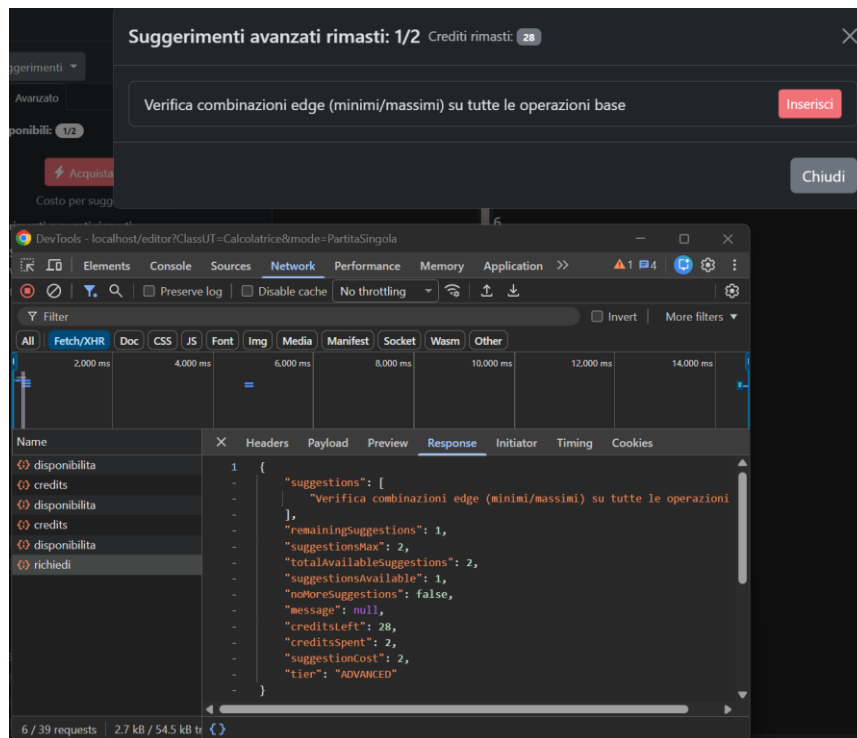
Passi:



1. Clicca “Richiedi suggerimento avanzato”.
2. Verifica risposta e saldo crediti aggiornato.

Risultato atteso:

- Suggerimento avanzato restituito.
- Crediti decrementati (spesa = 2).
- Contatore avanzati decrementato.



TS-SUG-07 — Richiesta suggerimento avanzato con crediti insufficienti

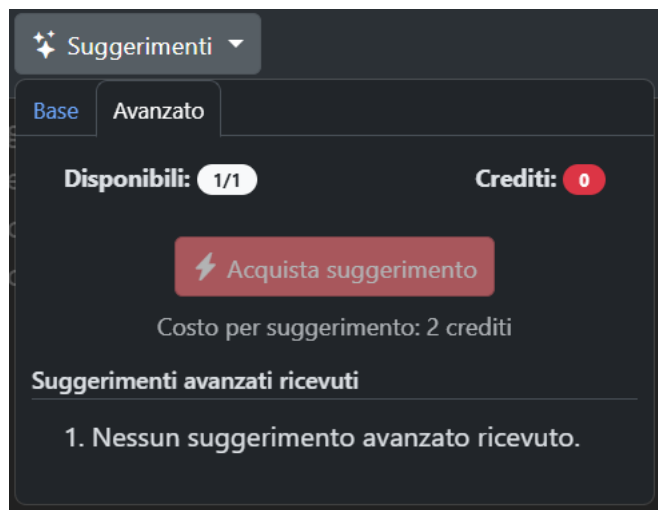
Precondizioni: crediti utente < 2; disponibilità avanzati > 0.

Passi:

Clicca “Richiedi suggerimento avanzato”.

Risultato atteso:

- UI mostra messaggio saldo crediti pari a 0 in rosso
- Pulsante di richiesta disabilitato, nessuna spesa crediti e nessun suggerimento consegnato.



TS-SUG-08 — Disponibilità avanzati esaurita

Precondizioni: nessun suggerimento avanzato disponibile per classe/difficoltà.

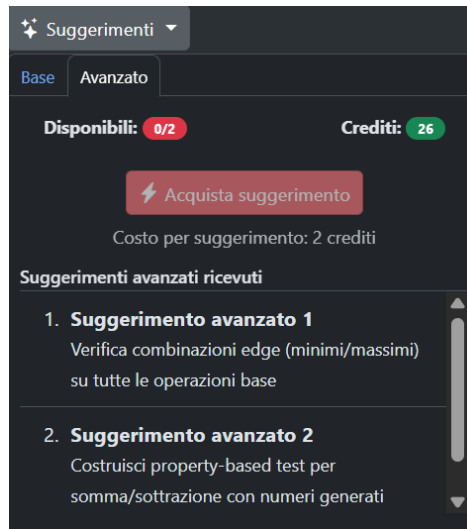
Passi:

1. Apri tab “Suggerimenti avanzati”.

2. Clicca “Richiedi suggerimento avanzato”.

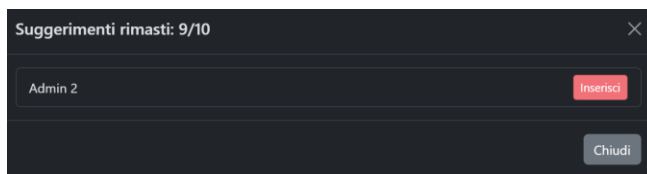
Risultato atteso:

- UI mostra contatore azzerato e pulsante richiesta suggerimento disattivato.
- Il pulsante disattivato inibisce la richiesta di ulteriori suggerimenti



TS-SUG-09 — Consistenza disponibilità tra refresh browser

Precondizioni: almeno 1 suggerimento già consegnato in partita.

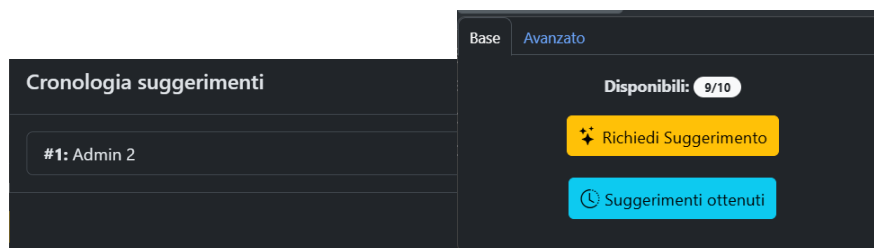


Passi:

1. Ricarica la pagina editor.
2. Attendi il ricalcolo disponibilità.

Risultato atteso:

- Il contatore riparte dal valore corretto (non si resetta a max).
- Lo storico si reidrata con i suggerimenti già ricevuti.

**TS-SUG-10 — Caricamento suggerimenti (admin)**

Precondizioni: account admin; inserimento suggerimento valido.

Passi:

1. Apri pagina admin suggerimenti.
2. Inserisci info suggerimento valide.
3. Esegui una richiesta suggerimento in partita per la stessa classe.

Risultato atteso:

- Suggerimento creato (201).

Inserisci nuovo suggerimento

Seleziona classe, difficoltà e tipologia, quindi aggiungi il testo del suggerimento.

Classi
FTPFile

Difficoltà
FACILE

Tipologia
AVANZATI

Lingua
it

Testo del suggerimento
Suggerimento Admin 1

Salva suggerimento Pulisci Torna alla dashboard

Suggerimento inserito correttamente.

Name	Headers	Payload	Preview	Response	Initiator	»
summary	▼ General					
create						
summary	Request URL	http://localhost/suggerimenti/admin/create				
	Request Method	POST				
	Status Code	201 Created				
	Remote Address	[::1]:80				
	Referrer Policy	strict-origin-when-cross-origin				

- Il suggerimento caricato è disponibile in partita.

✦ Suggerimenti ▼

Base Avanzato

Disponibili: 0/1 **Crediti:** 0

⚡ Acquista suggerimento

Costo per suggerimento: 2 crediti

Suggerimenti avanzati ricevuti

1. **Suggerimento avanzato 1**
Suggerimento Admin 1

TS-SUG-11 — Lista suggerimenti (admin)

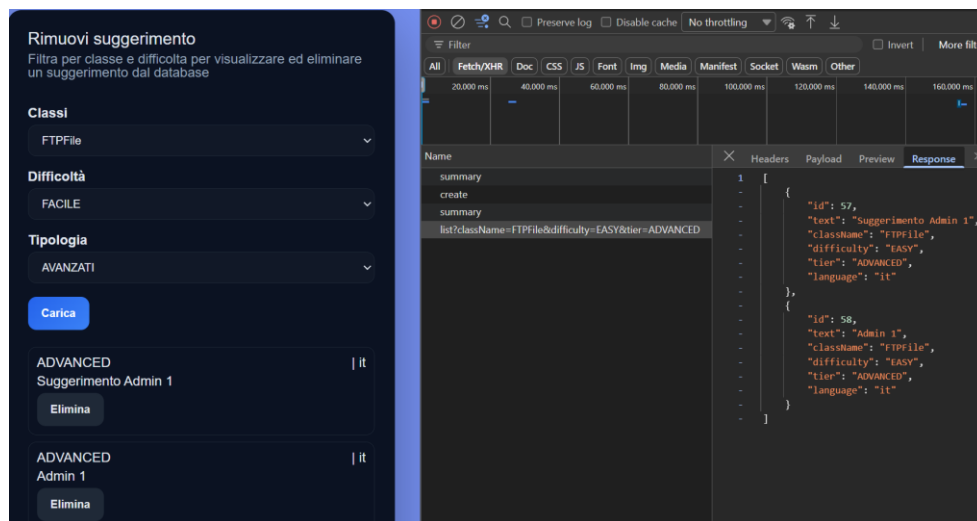
Precondizioni: suggerimenti presenti.

Passi:

1. Apri pagina admin suggerimenti.
2. Nella sezione di ricerca inserisci info classe valide.
3. Clicca su Carica

Risultato atteso:

- Chiamata `/suggerimenti/admin/list?className=...&difficulty=...`
- Lista restituita coerente con DB (testo, tier, difficulty, language).



TS-SUG-12 — Eliminazione suggerimento (admin)

Precondizioni: suggerimento esistente.

Passi:

1. Chiamare DELETE /suggerimenti/admin/{id}.

Risultato atteso:

- Suggerimento rimosso dal DB.

Name	✕	Headers	Preview	Response	Initiator	Timing	Cookies
summary		▼ General					
list?className=FTPFile&diff...		Request URL	http://localhost/suggerimenti/admin/1				
list?className=FTPFile&diff...		Request Method	DELETE				
list?className=Calcolatrice...		Status Code	● 204 No Content				
1		Remote Address	[::1]:80				
list?className=Calcolatrice...		Referrer Policy	strict-origin-when-cross-origin				
2							

6. Sviluppi Futuri e Raccomandazioni

Di seguito sono riportate alcune idee di sviluppi futuri da fare:

- Ottenimento crediti Tramite achievement
- Integrazione del sistema suggerimenti con LLM
- Aggiungere altre funzionalità per utilizzare i crediti ottenuti
- Aggiungere altri eventuali modi per guadagnare crediti
- Aggiungere lato amministratore import di file JSON di suggerimenti